Production-Grade Refactoring Blueprint

This document is a production-grade refactoring and optimization strategy for th
e Atho system. It is built from the two previous audit artifacts already produce
d in this repository:

- `Docs/Verification_Grade_Architecture_Audit.md`
- `Docs/Consensus_Verification_Grade_Audit.md`

It assumes the central engineering reality established by those audits:

- consensus, replay, reorg, public/private transaction handling, and bond/stake
eligibility are the most production-sensitive parts of the system
- the current codebase already contains substantial working machinery that shoul
d not be destabilized casually
- the main production problem is not “missing architecture,” but overlapping aut
hority, mixed responsibilities, inconsistent runtime assembly, insufficiently is
olated side effects, and not enough proof around the most dangerous state transi
tions
- the shortest path to production is not a rewrite; it is a phased transformatio
n that improves clarity, ownership, observability, and testability before it att
empts deep consensus surgery

This blueprint therefore does not recommend “refactor everything now.” It recomm
ends sequencing. The safe path is:

- freeze the consensus core conceptually before rewriting it
- reduce authority overlap outside consensus first
- make runtime ownership explicit
- split read paths from write paths
- wrap dangerous internals behind stable interfaces
- add the observability and test harnesses needed to prove behavior
- only then touch the highest-blast-radius consensus flows, and only if evidence
 still shows that they must change

The intended outcome is a codebase that is:

- easier to reason about
- safer to change
- easier to debug
- easier to test
- more deterministic under restart and reorg
- more modular where modularity reduces confusion
- still conservative around consensus-critical state

This is not a generic software-cleanup plan. It is a production-hardening roadma
p for a mixed public/private blockchain system with BPoW, bond/stake state, a GU
I, local services, shared LMDB state, and multiple runtime roles.

Current System Refactoring Readiness Assessment

Overall assessment

The system is refactorable, but not uniformly refactorable.

Some parts of the codebase are ready for immediate restructuring because they ar
e already failing the maintainability test without being the single source of ca
nonical truth. Other parts are not ready for deep redesign because they currentl
y carry too much consensus or replay responsibility and do not yet have enough o
bservability or regression coverage to survive invasive change safely.

The codebase does not look like an early toy project that needs “architecture ad

ded.” It looks like a system that has already accumulated architecture in severa
l layers at once:

- a consensus and block-admission layer
- a private transaction and proof layer
- a bond/stake and miner-eligibility layer
- a launcher and local process-control layer
- an API and service assembly layer
- a GUI that still acts as a partial orchestrator
- multiple storage-backed managers with partial rebuild and recovery semantics

That is an important distinction. The main refactoring challenge is not inventio
n. It is consolidation and risk management.

What kinds of refactors appear safe right now

The following categories appear safe enough to begin now, assuming they are done
 incrementally and with behavior-preserving wrappers first:

- file decomposition of oversized modules where the internal behavior is preserv
ed
- introduction of clearer interfaces around storage, service assembly, and trans
port
- centralization of error types and error codes
- structured logging and correlation IDs
- explicit state ownership documentation in code
- extraction of read-only query services away from mixed read/write modules
- GUI decomposition into view, controller, transport, and state-store modules
- API endpoint grouping and normalization without changing underlying business l
ogic
- control-plane cleanup that centralizes lifecycle authority without changing co
nsensus semantics
- repository wrappers around direct LMDB access in non-consensus paths
- revision markers and consistency metadata in read responses
- event-stream interfaces and backend event publishing, initially as additive be
havior

These are safe because they mostly improve clarity and observability without red
efining canonical truth.

What kinds of refactors appear dangerous right now

The following categories are dangerous right now and should not be treated as “c
leanup”:

- rewriting block acceptance sequencing
- changing the order in which block store, UTXO state, bond/stake state, and pri
vate state commit
- changing reorg rollback and replay semantics
- rewriting product-v1 private transaction validation
- changing how hidden/public conservation is enforced
- merging or replacing multiple validators without first proving rule equivalenc
e
- changing miner eligibility evaluation or bond activation logic
- changing fee-split, burn, or payout-set generation rules
- changing fork-choice semantics or tie-break behavior
- changing storage schema for canonical state without a replay-based migration a
nd rollback plan

These are dangerous because they affect consensus, replay, or production recover
y directly. Even if the current implementation is not ideal, destabilizing it be
fore stronger test and observability rails exist would increase risk rather than

reduce it.

Which areas seem fragile

The most fragile areas are the ones where canonical truth spans several modules and several stores at once:

- block acceptance plus state-hook application
- reorg execution plus extension-state rebuild
- private transaction validation plus proof gating plus hidden-supply accounting
- bond/stake replay plus payout-set generation plus settlement
- runtime role assembly where more than one graph may believe it owns the same stores
- GUI and API interaction where the frontend still infers or coordinates behavior that should be backend-owned

Fragility here does not mean the code is necessarily wrong. It means the blast radius of change is high because behavior emerges from interaction rather than from one clean owner.

Which areas seem structurally strong enough to build on

Several areas are strong enough to build around rather than replace:

- the existence of a `ConsensusSupervisor` coordinating block acceptance
- the existence of a dedicated `BlockVerifier`
- the existence of a dedicated `TransactionVerifier`
- the existence of replayable `BondStakeStore` and `PrivateNotesStore`
- the presence of explicit reorg journaling and rollback logic
- the presence of deterministic chainwork fork choice and deterministic tie-breaks
- the existence of tests that already encode some of the monetary and private-layer assumptions
- the use of explicit consensus write contexts inside several stores

This matters because the production path should preserve these strengths. The system does not need a blank-slate protocol engine. It needs clearer boundaries and safer operating conditions around the engines that already exist.

Where coupling is highest

Coupling appears highest in the following areas:

- `consensus.py`, where validation, orchestration, post-commit maintenance, slashing, replay recovery, and logging all intersect
- `gui.py`, where view logic, transport logic, node control, wallet flows, failover, and local state interpretation are mixed
- API bootstrap and service assembly, where transport, business logic, component instantiation, and lifecycle control are still too entangled
- node launcher/runtime assembly, where role processes, local API, P2P services, and storage-backed components are constructed in overlapping ways
- transaction handling, where there are multiple validation-related modules and it is not yet obvious to a maintainer which one is the true consensus path versus support path

High coupling is not just a style problem here. It increases the chance that a seemingly local refactor changes runtime authority, ordering, or state mutation paths unintentionally.

Where authority is most ambiguous

The previous audits already established the main ambiguity clusters, and they ar

e directly relevant to refactoring sequencing:

- node lifecycle control
- network switching
- API readiness and role readiness
- GUI displayed truth versus backend truth
- send-intent ownership and transaction lifecycle ownership
- mempool truth across process/runtime boundaries
- runtime assembly of consensus-capable components

Those areas should be targeted early, not because they are the most academically interesting, but because cleaning them up reduces the probability of accidental consensus-adjacent bugs everywhere else.

Where maintainability is worst

Maintainability appears worst where:

- files are very large and carry multiple domains
- side effects are hidden inside initialization paths
- transport and business logic are mixed
- several roles can instantiate the same core components
- the same state is reconstructed differently in different layers
- state changes are visible only indirectly through many log statements or follo
w-up repair paths

The worst offenders are likely:

- `Src/GUI/gui.py`
- API assembly and auth/bootstrap paths
- launcher/runtime assembly
- consensus orchestration paths

The problem is not simply file size. The problem is responsibility spread.

Where bug density is likely to be high

Bug density is likely highest where the code has all three of these properties:

1. high side-effect density
2. high branching and error-recovery complexity
3. weak isolation between domains

By that measure, likely high-bug-density areas are:

- network switching
- startup and restart sequencing
- send transaction lifecycle and retry behavior
- GUI/backend synchronization and stale-state handling
- private transaction proof and transformation paths
- reorg plus replay plus post-commit recovery paths

Those areas deserve better observability before they receive deeper refactors.

Where code size and responsibility spread are warning signs

The warning signs are not evenly distributed. The strongest warnings are:

- giant files acting as application layers instead of modules
- lifecycle logic implemented in multiple places
- duplicated validation concepts across several modules
- storage repositories not yet cleanly separated from higher-level services

- query logic mixed with write or orchestration logic
- frontend logic still making backend-ownership decisions

The main takeaway is that the codebase is ready for decomposition and wrapper extraction, but not for broad “simplification by rewrite.”

What parts should be stabilized before being deeply redesigned

The following must be stabilized before any deeper redesign:

- consensus observability
- replay and reorg verification coverage
- runtime single-writer assumptions
- startup/readiness tracing
- transaction lifecycle instrumentation
- payout-set and bond/stake replay equivalence tests
- private proof-path reliability

This is especially important because the user’s stated priority is consensus that is effectively production-safe. That means the roadmap should treat consensus as the protected center, not the first target for aesthetic cleanup.

Refactoring-readiness conclusion

The system is ready for:

- boundary cleanup
- ownership cleanup
- decomposition of large modules
- service extraction for non-consensus paths
- observability work
- API and GUI normalization
- control-plane centralization

The system is not yet ready for:

- consensus rewrites
- replay model rewrites
- storage-order rewrites
- private proof model rewrites

The correct strategy is to make the dangerous areas easier to see and easier to test before changing their internals.

Refactoring-readiness by subsystem

The readiness picture is not uniform, and the roadmap should reflect that explicitly.

Consensus core

Readiness level:

- low for invasive refactors
- medium for observability and wrapper extraction

Why:

- canonical truth flows through it
- replay and reorg equivalence is not yet fully proven
- private/bond state hooks make change impact non-local

Safe actions:

- tracing
- invariant documentation
- façade extraction around acceptance results

Unsafe early actions:

- sequencing rewrites
- commit-order changes
- validator unification for aesthetic reasons

Mempool and transaction lifecycle

Readiness level:

- medium

Why:

- admission ownership is important but can be normalized behind a service boundary
- many problems here are product/control-plane problems rather than protocol changes

Safe actions:

- lifecycle records
- result normalization
- service façade extraction

Riskier actions:

- changing admission semantics
- changing miner-facing mempool selection before ownership is clean

Wallet flows

Readiness level:

- medium-high for façade extraction
- low for secret-lifetime redesign unless carefully traced

Why:

- most wallet pain comes from scattered ownership rather than deep cryptographic redesign

Safe actions:

- `WalletService` façade
- preview and lifecycle normalization
- unlock-state contract cleanup

Riskier actions:

- key-material lifetime redesign
- deep wallet storage changes

GUI

Readiness level:

- high for structural decomposition

Why:

- it is large, coupled, and error-prone
- it is not the owner of canonical truth and therefore can be refactored more safely than backend consensus paths

Safe actions:

- file splitting
- state-store extraction
- transport/client cleanup

Riskier actions:

- event-driven behavior changes before backend contracts are stable

API

Readiness level:

- high for transport/service separation

Why:

- its current ambiguity is a major architectural problem
- it can be improved substantially without changing consensus rules

Safe actions:

- response normalization
- command/query split
- auth cleanup
- route extraction

Riskier actions:

- removing old endpoints too quickly
- changing command semantics without idempotency support

Control plane

Readiness level:

- medium-high

Why:

- it is one of the biggest sources of authority confusion
- but process-control bugs can affect the whole runtime

Safe actions:

- supervisor façade
- readiness model
- health and restart tracing

Riskier actions:

- forcing hard single-writer locks before topology is fully understood

Storage

Readiness level:

- medium for repository extraction
- low for deep canonical-write redesign

Why:

- wrappers are high value
- changing write semantics is dangerous

Safe actions:

- repository reads
- revision markers
- schema metadata

Riskier actions:

- commit-order redesign
- engine replacement

P2P and sync

Readiness level:

- medium-low

Why:

- network behavior is important, but not the shortest path to reducing overall p
roduction risk

Safe actions:

- peer store interfaces
- sync status surfaces
- transport tracing

Riskier actions:

- broad synchronization redesign
- aggressive concurrency changes

This subsystem view is important because it turns the roadmap into a selective e
ngineering program rather than a blanket refactor mandate.

Refactoring Principles for This Codebase

Principle 1: Preserve consensus correctness above all else

Any refactor that can change:

- block validity
- transaction validity
- fork choice
- replay results
- reorg results
- hidden/public conservation
- bond/stake eligibility

must be treated as a protocol change until proven otherwise. Even if the code diff looks like a cleanup, it is not a low-risk cleanup if it can change any of those behaviors.

This principle should dominate every tradeoff. Maintainability is important, but not at the cost of ambiguous consensus behavior.

Principle 2: Reduce overlapping authority before optimizing performance

The system's current risk is not primarily that it is too slow. The system's current risk is that too many layers still participate in deciding or interpreting state:

- GUI interprets too much
- API assembles too much
- launcher and node controller overlap
- several runtimes can instantiate consensus-capable graphs

Performance work done before authority cleanup often locks in the wrong boundaries. The roadmap should therefore prefer:

- cleaner ownership
- clearer service boundaries
- explicit lifecycle authority

before throughput or latency work that would make these boundaries harder to change.

Principle 3: Centralize canonical state mutation

Canonical state mutation must happen through the smallest possible number of authoritative paths.

That means:

- one block-acceptance path
- one transaction-admission path
- one lifecycle authority for starting and stopping runtimes
- one network-switch authority
- one authoritative backend transaction lifecycle

Other layers may observe or request changes, but they should not mutate canonical or operational truth directly.

Principle 4: Isolate side effects

The current codebase often mixes:

- computation
- validation
- persistence
- process control
- logging
- network I/O
- user-facing retries

inside the same function or module.

The safest refactor pattern is to separate:

- pure logic from mutation

- mutation from orchestration
- orchestration from transport

This is how the system becomes easier to test and easier to reason about.

Principle 5: Move from implicit coupling to explicit interfaces

A large part of the present complexity comes from components knowing too much about each other's internals or construction patterns. Refactoring should introduce interfaces where they reduce confusion:

- repositories for storage access
- services for write ownership
- query services for read models
- gateway interfaces for transport boundaries
- event interfaces for state propagation

The purpose is not abstraction for its own sake. The purpose is to make dependencies visible and stable.

Principle 6: Split read paths from write paths

Read-heavy code should not be mixed with mutation-heavy consensus code unless there is a clear reason.

This matters because:

- read models can tolerate caching and composition strategies that writes cannot
- write paths need explicit authority and deterministic sequencing
- GUI and explorer concerns are mostly read-model problems, not chainstate-owner problems

Separating them reduces both performance interference and conceptual interference.

Principle 7: Prefer wrappers before invasive rewrites

Where a subsystem is ugly but operationally important, the first move should usually be:

- wrap it
- instrument it
- normalize its public surface
- add tests around it

Only after that should deeper surgery happen.

This is especially true for:

- consensus orchestration
- replay and reorg handling
- LMDB-backed stores
- send transaction lifecycle

Principle 8: Improve observability before deep surgery

When a subsystem is fragile, changing it blind is dangerous.

Before deeper refactors, add:

- correlation IDs
- structured logs

- state transition logs
- recovery tracing
- branch/reorg tracing
- transaction lifecycle tracing
- startup dependency tracing

Observability is not optional overhead here. It is part of the safety rail.

Principle 9: Make lifecycle ownership explicit

At present, lifecycle ownership is too distributed. Refactoring should force the codebase to answer clearly:

- who starts roles
- who stops roles
- who decides readiness
- who owns restart policy
- who performs network switch orchestration

This should converge on a supervisor-style authority, with the GUI becoming a client of lifecycle APIs rather than a fallback orchestrator.

Principle 10: Keep the consensus core coarse-grained until correctness is proven

There is a common mistake in refactoring ambitious systems: splitting the hardest code into many small components too early.

For Atho, some parts should stay relatively coarse-grained until they are better proven:

- block acceptance
- reorg and replay
- mixed public/private validity
- bond/stake replay and settlement

The goal is not microservices. The goal is explicit boundaries and safe internal structure.

Principle 11: Separate convenience logic from canonical truth

A lot of pain in the current architecture comes from convenience logic leaking into state interpretation:

- GUI inference
- API caches with weak revision semantics
- transport assembly mixed with runtime authority

The refactor philosophy should be:

- convenience logic may observe
- convenience logic may cache
- convenience logic may format
- convenience logic may never redefine or infer canonical truth when it should be receiving it from an owner

Principle 12: Make transaction and block lifecycles explicit

Ambiguous lifecycles create bugs. The system should move toward explicit lifecycle models for:

- incoming tx

- pending tx
- locally accepted tx
- relayed tx
- included tx
- reorged-out tx
- confirmed tx
- failed tx

and for blocks:

- received
- verified
- orphaned
- side branch
- canonical extend
- reorg-applied
- rollback-restored

This improves correctness, debugging, and operator understanding.

Principle 13: Introduce service boundaries only where they reduce confusion

Not every module deserves to become a service. Service boundaries should be intr
oduced where they:

- reduce authority ambiguity
- remove mixed responsibilities
- improve testability
- create cleaner control-plane ownership
- protect consensus code from product-layer churn

They should not be introduced merely to make diagrams look cleaner.

Principle 14: Production safety beats architectural purity

If the cleanest theoretical architecture would require destabilizing replay or b
lock acceptance before the surrounding system is better instrumented, it should
wait.

This blueprint is intentionally conservative. Production-grade in a blockchain c
ontext means:

- fewer surprises
- narrower mutation boundaries
- easier rollback
- stronger proof

not maximum elegance per line of code.

Principles conclusion

The philosophy of this refactoring program is:

- protect consensus
- clean the boundaries around consensus first
- make ownership obvious
- move inference and orchestration out of the wrong layers
- prefer additive safety rails before subtractive rewrites
- only optimize after the architecture is easier to trust

That philosophy drives the priority map below.

Anti-patterns this roadmap explicitly rejects

The roadmap is designed around several things the team should not do, even if th
ey look attractive in the short term.

Anti-pattern 1: “Clean architecture by big rewrite”

Rejected because:

- it destroys rollback safety
- it hides which behavior changes were intentional
- it is almost guaranteed to over-touch consensus-adjacent code

Preferred alternative:

- wrappers, incremental extraction, differential testing

Anti-pattern 2: “Put everything behind the API”

Rejected because:

- it turns the API into a second runtime owner
- it encourages transport-layer business logic
- it makes local service testing harder

Preferred alternative:

- services own business logic, API owns transport

Anti-pattern 3: “Split consensus into many tiny pieces immediately”

Rejected because:

- it increases indirection before the rules and write order are better proven
- it can create false clarity while actually hiding state transitions

Preferred alternative:

- keep the core coarse-grained, improve seams and traces first

Anti-pattern 4: “Optimize away the complexity”

Rejected because:

- performance work does not solve ambiguous authority
- optimized confusion is still confusion

Preferred alternative:

- fix ownership and observability first, then optimize

Anti-pattern 5: “Let the GUI smooth over backend ambiguity”

Rejected because:

- it shifts canonical or operational interpretation into the least appropriate l
ayer
- it makes bugs harder to reproduce and diagnose

Preferred alternative:

- backend-owned lifecycle, query consistency, and event models

Anti-pattern 6: “Introduce services without reducing coupling”

Rejected because:

- renaming a module to `SomethingService` does not improve architecture if it still exposes internals or shares mutation authority

Preferred alternative:

- real ownership boundaries, narrow interfaces, and forbidden dependency rules

System-Wide Refactoring Priority Map

Priority buckets

This map classifies work into five buckets:

- `Do First`
- `Do Soon`
- `Do Later`
- `Do Last / High Risk`
- `Avoid Unless Necessary`

The buckets are based on:

- production value
- blast radius
- dependencies
- rollback difficulty
- consensus risk

Priority map table

Area	Bucket	Why it belongs there	Expected value	Expected risk	Dependencies	Must be true before touching
Observability and tracing	Do First	prerequisite for safe deeper refactors	very high	low	none	none
Error normalization and response envelopes	Do First	lowers ambiguity fast without touching consensus	high	low	API organization	basic endpoint inventory
GUI decomposition and state-store extraction	Do First	removes orchestration from frontend and reduces bug surface	high	low-medium	observability helpful but not required	preserve current behaviors behind wrappers
API command/query separation	Do First	clarifies ownership and read/write boundaries	high	low-medium	endpoint inventory	no consensus behavior change
Launcher/supervisor ownership cleanup	Do First	fixes major runtime authority ambiguity	very high	medium	basic process tracing	understand current role startup graph
Query/read model extraction	Do First	reduces hot-path coupling and GUI confusion	high	low	API organization	revision metadata design
Storage repository wrappers for non-consensus paths	Do First	removes direct LMDB spread and unlocks safer refactors	high	low-medium	interface design	do not alter consensus mutation order
Transaction lifecycle normalization	Do Soon	improves send stability and GUI/backend consistency	high	medium	eventing and error normalization	explicit backend lifecycle model
WalletService extraction	Do Soon	reduces API and GUI coupling	high	medium	command/query split	clear service interfaces
NodeService centralization	Do Soon	centralizes runtime mutations and autho

rity | very high | medium | supervisor design | role ownership model defined |
Event bus / backend event stream	Do Soon	reduces polling and stale-state behavior	high	medium	lifecycle/event model	correlation IDs and revision markers
Explorer/query bridge isolation	Do Soon	reduces query load and display coupling	medium-high	low-medium	query service	API grouping
Mempool service boundary cleanup	Do Soon	clarifies tx admission ownership without rewriting consensus	medium-high	medium	tx lifecycle model	validation ownership mapped
P2P stack cleanup	Do Later	important but higher runtime blast radius	medium	medium-high	supervisor cleanup	better process tracing and tests
Sync engine cleanup	Do Later	valuable after authority cleanup	medium	medium-high	P2P boundaries clearer	replay and state tests stronger
Storage schema/versioning framework	Do Later	needed for long-term production upgrades	high	medium-high	repository wrappers	migration policy ready
Performance optimization	Do Later	worthwhile only after boundaries stabilize	medium-high	medium	observability and profiling	architecture cleaner
Consensus module decomposition	Do Last / High Risk	high blast radius, protocol-sensitive	high	very high	extensive tests and tracing	replay equivalence proven
Reorg/replay model changes	Do Last / High Risk	deepest state risk in codebase	high	very high	strong reorg harness	live apply vs replay equivalence proven
Private tx validation path changes	Do Last / High Risk	hidden-inflation and proof risk	very high	very high	proof harness	failing proof tests explained/fixed
Bond/stake settlement logic changes	Do Last / High Risk	touches miner eligibility, payouts, and replay	high	very high	settlement tests	full payout invariant suite
Fork choice changes	Avoid Unless Necessary	stable deterministic rule already exists	low incremental value	extreme	none	only touch if correctness issue proven
Monetary policy rule changes	Avoid Unless Necessary	protocol-critical and already partly tested	low architectural value	extreme	none	only touch if bug proven
Deep LMDB replacement	Avoid Unless Necessary	large migration cost with weak short-term value	low-medium	extreme	storage wrappers, migrations, benchmarks	current backend proven insufficient

Area-by-area explanation

Consensus

Bucket: `Do Last / High Risk`

Why:

- It is the center of production risk.
- The previous audits show that the consensus core already contains real, nontrivial machinery.
- The biggest current consensus problem is not absence of code. It is proof, visibility, and boundary clarity around that code.

Expected value:

- High if truly needed.
- Negative if attempted prematurely.

What must be true first:

- stronger observability

- replay equivalence tests
- refactored outer layers that stop interfering with it

Block acceptance flow

Bucket: `Do Last / High Risk`

Why:

- It already sequences block verify, tx verify, emission checks, payout checks, commit, state-hook application, and post-commit cleanup.
- Any change here can alter canonical truth.

What should happen first instead:

- instrument it
- document it
- wrap its outputs
- build invariant and replay tests around it

Transaction pipeline

Bucket: `Do Soon`

Why:

- It has ownership ambiguity outside the canonical validator path.
- Send-intent and lifecycle ambiguity create product bugs.
- Much of the improvement can happen by wrapping and normalizing lifecycle state without changing the core validity engine.

Private tx pipeline

Bucket: `Do Last / High Risk`

Why:

- It is directly tied to hidden/public conservation and proof validation.
- The prior audit already found test failures in this area.

Safe preparatory work:

- tracing
- better interfaces
- test harnesses
- clear isolation between transformation, proof statement building, and validation

Staking/bonding logic

Bucket: `Do Last / High Risk`

Why:

- It affects miner eligibility, slashing, payout planning, and replay.

Safe nearby work:

- surface cleanup
- snapshot/read service extraction
- instrumentation

P2P stack

Bucket: `Do Later`

Why:

- It matters, but the biggest current risk is not protocol propagation speed. It is authority overlap and runtime assembly ambiguity.
- Cleaning control-plane ownership first will make P2P cleanup safer.

Sync engine

Bucket: `Do Later`

Why:

- Similar to P2P. Valuable, but not the shortest path to production safety right now.

Mempool

Bucket: `Do Soon`

Why:

- Admission ownership and lifecycle semantics need cleanup.
- Much of that can be done by introducing a `MempoolService` boundary without changing the underlying store immediately.

Storage layer

Bucket: `Do First` for wrappers, `Do Later` for deeper schema/versioning work

Why:

- Direct LMDB access spread is a maintainability problem.
- But canonical store order and semantics are too risky to rewrite immediately.

Wallet layer

Bucket: `Do Soon`

Why:

- Wallet concerns are mixed into GUI, API, and runtime assembly.
- Extracting a `WalletService` is high value and medium risk if done incrementally.

API layer

Bucket: `Do First`

Why:

- It is one of the major authority-confusion zones.
- Many improvements are structural and transport-oriented rather than protocol-changing.

GUI layer

Bucket: `Do First`

Why:

- `gui.py` is oversized
- frontend orchestration is too heavy
- decomposing it will reduce bug density and make backend ownership easier to en force

Launcher / supervisor logic

Bucket: `Do First`

Why:

- It is a top-tier authority ambiguity problem.
- Centralizing it reduces runtime overlap risk across the whole system.

Event/update propagation

Bucket: `Do Soon`

Why:

- Polling and stale-state issues are product problems with architecture implicat ions.
- Eventing reduces GUI guesswork and helps observability.

Explorer/query systems

Bucket: `Do First`

Why:

- Query extraction is comparatively safe and high leverage.

Logging/observability

Bucket: `Do First`

Why:

- prerequisite for nearly every medium- or high-risk refactor

Tests

Bucket: `Do First`

Why:

- test debt is what makes the dangerous work dangerous

Performance optimization

Bucket: `Do Later`

Why:

- early optimization will harden the wrong boundaries

Deployment/packaging

Bucket: `Do Soon`

Why:

- production readiness includes lifecycle, upgrades, config safety, and restore operations
- but packaging should follow control-plane cleanup, not precede it

Priority-map conclusion

The system-wide order is clear:

1. observability, tests, and ownership clarity
2. GUI/API/control-plane cleanup
3. service and repository extraction
4. runtime topology cleanup
5. only then consider consensus-sensitive changes

That sequencing is the core production-safety strategy.

Least Risky, Highest Value Refactors First

1. Introduce a clear system ownership map in code

Exact change:

- Add a lightweight architecture ownership document in the repo and align code comments/docstrings in the relevant services.
- Explicitly name the owner of:
  - block acceptance
  - tx admission
  - node lifecycle
  - network switch
  - wallet send intent
  - GUI state
  - API readiness

Why it is low risk:

- no behavior change

What it improves:

- eliminates ambiguity for future refactors
- reduces accidental boundary crossing

What it unlocks:

- service extraction without ownership confusion

2. Add revision markers and consistency metadata to backend reads

Exact change:

- Every API query response that composes state should include:
  - request id
  - chain or state revision
  - tip height
  - tip hash
  - network
  - node role
  - source cache or freshness info

Why it is low risk:

- additive response fields only

What it improves:

- stale-read diagnosis
- GUI consistency handling
- debugging of race conditions

What it unlocks:

- revision-aware GUI state store
- event-stream reconciliation

3. Centralize error taxonomy

Exact change:

- define stable backend error codes and categories:
  - retryable
  - invalid input
  - stale state
  - auth required
  - conflict
  - backend unavailable
  - consensus reject
  - lifecycle not ready

Why it is low risk:

- can be introduced as wrappers around existing errors first

What it improves:

- GUI behavior
- logging clarity
- operational debugging

What it unlocks:

- less string-matching in GUI
- safer retries

4. Split giant files without changing behavior

Exact change:

- split `gui.py` into module files behind the same public entrypoint
- split API routing from API component assembly
- split launcher CLI entrypoints from process control helpers

Why it is low risk:

- file movement and import cleanup only if done carefully

What it improves:

- maintainability
- reviewability
- local reasoning

What it unlocks:

- targeted unit tests
- isolated service refactors

5. Extract read-only query services

Exact change:

- create dedicated query/read modules for:
  - chain summary
  - balance/history views
  - peer/status views
  - explorer/index queries

Why it is low risk:

- no canonical writes are moved

What it improves:

- reduces API/business-logic mixing
- reduces GUI composition complexity

What it unlocks:

- command/query separation
- caching discipline

6. Add correlation IDs and structured transition logs

Exact change:

- every request, send intent, block acceptance, reorg, startup sequence, and network switch gets a correlation id

Why it is low risk:

- additive observability only

What it improves:

- debugging
- production diagnosis
- confidence for later refactors

What it unlocks:

- phase-gated consensus-sensitive work

7. Wrap lifecycle control behind one service surface

Exact change:

- introduce a `SupervisorService` facade even if internally it still calls current launcher/controller code

Why it is low risk:

- can delegate to current behavior first

What it improves:

- ownership clarity
- GUI/backend contract

What it unlocks:

- real lifecycle centralization later

8. Make GUI a stateful client rather than a retry script

Exact change:

- introduce one GUI state store and move transport/retry behavior out of widget code

Why it is low risk:

- can be done as file splits and routing through the store while preserving screens

What it improves:

- frontend consistency
- reduced race conditions

What it unlocks:

- event-driven updates
- less local authority inference

9. Introduce explicit backend readiness contracts

Exact change:

- define a typed readiness model that distinguishes:
  - process live
  - backend initialized
  - recovery complete
  - network identity confirmed
  - wallet service ready
  - consensus service ready
  - command classes allowed
- expose that model consistently through status endpoints and lifecycle events

Why it is low risk:

- it formalizes existing concepts rather than changing consensus behavior

What it improves:

- safer GUI gating
- cleaner restart behavior
- easier support diagnosis

What it unlocks:

- removal of GUI-side readiness guesses
- stricter supervisor orchestration later

10. Label every endpoint as command, query, admin, or debug

Exact change:

- inventory the API surface and tag each endpoint by intent and safety class
- distinguish:
  - read-only safe local queries
  - privileged wallet commands
  - lifecycle/admin commands
  - trace and debug surfaces

Why it is low risk:

- classification and routing metadata only

What it improves:

- auth cleanup
- clearer client usage
- easier deprecation planning

What it unlocks:

- rate limiting by endpoint family
- explorer/public exposure hardening

11. Add runtime guard assertions around ownership boundaries

Exact change:

- add lightweight runtime assertions where safe to detect:
  - GUI invoking forbidden local mutation helpers
  - API routes instantiating forbidden consensus objects
  - duplicate lifecycle owners attempting the same action
  - unexpected direct repository writes from product layers

Why it is low risk:

- initially warn-only or debug-only

What it improves:

- catches architectural drift early
- turns implicit rules into enforceable rules

What it unlocks:

- safer multi-contributor refactors
- stronger confidence in single-authority service adoption

12. Normalize configuration loading and network identity resolution

Exact change:

- centralize config parsing and network identity derivation behind one module co
nsumed by CLI, API, supervisor, and GUI

Why it is low risk:

- can be introduced as a shared helper without changing runtime semantics

What it improves:

- reduces mixed-network bugs
- reduces duplicated environment and file parsing logic
- simplifies upgrade and migration work later

What it unlocks:

- cleaner network-switch orchestration
- safer packaging and installer behavior

13. Create compatibility shims for legacy entrypoints

Exact change:

- keep existing scripts and route surfaces functioning, but make them delegate to new services or wrappers

Why it is low risk:

- minimizes operational disruption while refactors land

What it improves:

- rollback safety
- operator continuity
- incremental migration feasibility

What it unlocks:

- phased refactors without “big bang” cutovers

14. Standardize internal result objects for domain actions

Exact change:

- introduce typed result objects for:
  - tx admission
  - block acceptance
  - lifecycle command execution
  - wallet send preview
  - network-switch attempt

Why it is low risk:

- wrappers can translate current return values into normalized results without altering behavior

What it improves:

- predictable error handling
- easier API normalization
- cleaner tests

What it unlocks:

- event publishing from a consistent model
- easier metrics and tracing

15. Add invariant documentation adjacent to code, not just in papers

Exact change:

- place short invariant notes in or beside the modules that own:
  - consensus acceptance
  - replay/recovery
  - private-state updates

  - bond/stake state
  - lifecycle authority
  - query consistency

Why it is low risk:

- documentation-only or comment-level initially

What it improves:

- reduces maintenance drift between intent and implementation
- makes reviews more precise

What it unlocks:

- runtime assertions and targeted tests aligned with the documented invariants

Low-risk refactor conclusion

The low-risk program is larger than simple cleanup. It is a deliberate campaign to make the system legible. By the end of this stage, maintainers should be able to answer who owns a mutation, which revision a screen is showing, and which service is allowed to perform an operational action. That clarity is the platform on which the medium- and high-risk work depends.

Medium-Risk Structural Refactors

1. Extract a real `WalletService`

Current problem:

- wallet behavior is spread across GUI flows, API handlers, node assembly, `Src/Main/send.py`, `Src/Wallet/private_spend.py`, `Src/Wallet/private_receive.py`, and transaction-building utilities
- wallet unlock, send intent, address management, preview generation, and submission are not yet cleanly owned by one backend layer
- the frontend still appears to carry too much knowledge about retry, status interpretation, and local fallback behavior

Target state:

- `WalletService` becomes the single backend owner for:
  - key and account operations
  - unlock and lock state
  - send intent creation
  - public tx build requests
  - private tx build requests
  - transaction preview and confirmation data
  - submission requests into `MempoolService` or `NodeService`
- API becomes transport only
- GUI becomes a client of wallet commands and wallet events

Migration strategy:

1. Introduce a façade class that delegates to existing wallet-related functions.
2. Move endpoint handlers to call the façade instead of directly reaching lower-level functions.
3. Move GUI send flows to call the normalized API only.
4. Add a stable transaction-lifecycle record returned from wallet commands.
5. Only after the façade is stable, move internal functions and state behind the service boundary.

Test requirements:

- wallet unlock and lock contract tests
- idempotent send-intent creation tests
- public send preview and submit tests
- private send preview and submit tests
- send failure classification tests
- restart-during-send tests

Rollback difficulty:

- medium, because API and GUI will both start depending on the new surface
- low if the first phase is only a delegation façade

Why it is medium risk:

- it changes ownership and call paths
- it should not change consensus rules or mempool rules if implemented correctly
- the risk is primarily lifecycle ambiguity and regression in user-facing flows, not canonical chain truth

2. Introduce an actual `NodeService`

Current problem:

- runtime mutation responsibilities are spread across `Src/Main/runnode.py`, `Src/Main/stop.py`, `Src/Node/walletnode.py`, `Src/Node/minernode.py`, API bootstrap code, and local control helpers
- several paths can instantiate consensus-capable components or interact with storage-backed managers
- readiness, status, and role ownership are not sufficiently centralized

Target state:

- `NodeService` owns:
  - local node runtime handles
  - submission of commands into consensus-capable backends
  - status snapshots for role processes
  - node-scoped operations that are not pure transport
- `SupervisorService` owns process lifecycle
- `NodeService` never becomes a GUI helper or a transport layer

Migration strategy:

1. Define a canonical `NodeService` interface.
2. Route API node endpoints to this interface.
3. Make the service initially delegate to existing launch/control paths.
4. Add explicit status models for process state, readiness state, network state, and active role set.
5. Remove direct runtime control from GUI once the service proves stable.

Test requirements:

- start, stop, restart contract tests
- role-status consistency tests
- network-switch sequencing tests
- readiness versus liveness tests

Rollback difficulty:

- medium
- process control changes can be reverted if the original entrypoints remain cal

lable underneath the new façade

Why it is medium risk:

- runtime ownership changes affect the whole product
- however, if the service delegates to current process controls at first, the risk remains operational rather than consensus-critical

3. Separate `APIGateway` from backend assembly

Current problem:

- the API layer still acts like transport, dependency assembler, business-logic surface, readiness interpreter, and at times a runtime bootstrapper
- that produces authority confusion and makes it harder to test backend behavior independently of HTTP

Target state:

- `APIGateway` owns:
  - HTTP transport
  - auth and scope enforcement
  - request validation
  - response normalization
  - idempotency keys for commands
  - versioning and rate limiting
- it calls backend services only
- it does not construct consensus or wallet runtimes directly

Migration strategy:

1. Split route definitions from service wiring.
2. Introduce a service container that receives already-built services from the runtime.
3. Move business logic out of route functions into backend services.
4. Introduce command/query endpoint families and response envelopes.
5. Freeze the API surface and deprecate old route behavior gradually.

Test requirements:

- API contract tests
- auth-scope tests
- stale-read versus fresh-read response tests
- idempotent command retry tests

Rollback difficulty:

- medium
- mostly limited to transport/business-logic boundaries

Why it is medium risk:

- many callers depend on the API layer
- but the protocol semantics do not change if the extracted services preserve behavior

4. Introduce a real `QueryService`

Current problem:

- read paths are too mixed with write paths
- GUI and explorer behavior likely read across multiple endpoints and possibly m

ultiple revisions
- there is no strong boundary between convenience view composition and canonical mutation layers

Target state:

- `QueryService` owns:
  - derived read models
  - balance/history composition
  - explorer-facing summaries
  - status dashboards
  - peer and sync snapshots
- it is read-only
- it may use caches, but every cache must expose freshness metadata

Migration strategy:

1. Identify existing read-heavy endpoints and group them.
2. Create view-model functions that compose existing read logic without changing underlying writes.
3. Add revision metadata to every response.
4. Move GUI screens to consume these view models instead of stitching state from many low-level calls.

Test requirements:

- snapshot consistency tests
- revision marker tests
- cross-endpoint consistency tests
- large-history query performance tests

Rollback difficulty:

- low to medium
- query surfaces can coexist with existing endpoints during migration

Why it is medium risk:

- it changes API and GUI behavior materially
- but it should not alter canonical writes if the read model remains read-only

5. Formalize a backend `EventBus`

Current problem:

- GUI/backend synchronization is too polling-heavy
- state changes are too often inferred from repeated reads
- lifecycle, send, sync, and reorg events do not yet appear to have one normalized stream

Target state:

- backend emits typed events for:
  - node started/stopped/restarting
  - sync progress
  - new tip
  - reorg applied
  - mempool delta
  - wallet balance changed
  - wallet history changed
  - transaction lifecycle updated
  - network switched

  - backend degraded/recovering
- GUI consumes the event stream and reconciles it against revision-aware queries

Migration strategy:

1. Add event production alongside existing logs.
2. Expose an additive local event endpoint or stream.
3. Add GUI adapters that consume the events but still fall back to polling.
4. Move screens gradually from polling-first to event-first.

Test requirements:

- event ordering tests
- event loss recovery tests
- replay/reconnect tests
- revision reconciliation tests

Rollback difficulty:

- medium
- once the GUI depends heavily on events, rollback becomes harder

Why it is medium risk:

- event models affect many surfaces at once
- if designed badly they can amplify inconsistency
- but if designed with revision-aware reconciliation, they dramatically improve stability

6. Normalize transaction lifecycle ownership

Current problem:

- send intent, build status, local acceptance, relay state, confirmation state, and failure state are not obviously owned by one backend domain object
- GUI likely infers too much from individual endpoints or ad hoc responses

Target state:

- `WalletService` creates a `TransactionIntent`
- `NodeService` or `MempoolService` reports admission and relay state
- `QueryService` exposes lifecycle snapshots
- `EventBus` emits lifecycle transitions
- the GUI renders a lifecycle model instead of guessing

Migration strategy:

1. Add a lifecycle record keyed by client request id or backend intent id.
2. Map current send flows to lifecycle states.
3. Expose lifecycle query and lifecycle events.
4. Remove GUI inference once backend transitions are complete.

Test requirements:

- duplicate submit tests
- restart-mid-lifecycle tests
- reorg-out transaction lifecycle tests
- relay failure and delayed confirmation tests

Rollback difficulty:

- medium

- depends on how deeply the GUI consumes the new lifecycle state

Why it is medium risk:

- it crosses wallet, mempool, node, API, and GUI boundaries
- but it can be introduced as a new layer without changing consensus logic

7. Extract `PeerManager` and sync-control surfaces cleanly

Current problem:

- peer logic currently lives across `Src/Network/peers.py`, `Src/Network/sync.py`, `Src/Storage/peerstore.py`, and node runtime layers
- peer truth, scoring, banning, and sync decisions are not yet isolated behind one clear owner

Target state:

- `PeerManager` owns:
  - peer records
  - peer scoring and ban state
  - selection of peers for sync/download policies
- `SyncService` owns:
  - chain synchronization workflow
  - download scheduling
  - tip catch-up state
- consensus validates blocks; peer code never defines validity

Migration strategy:

1. Add interfaces around peer store reads and writes.
2. Move peer selection logic behind a manager class.
3. Keep existing network transport intact while redirecting control decisions through the manager.
4. Only later clean up transport-level code.

Test requirements:

- peer scoring tests
- ban/unban persistence tests
- sync peer selection tests
- restart persistence tests

Rollback difficulty:

- medium

Why it is medium risk:

- transport and persistence will be touched
- but core consensus rules remain unaffected if block acceptance ownership stays unchanged

8. Isolate explorer and web-bridge concerns

Current problem:

- explorer behavior and GUI/web bridge concerns risk pulling query traffic and presentation assumptions into the main application layers

Target state:

- explorer-facing logic consumes `QueryService`
- no explorer code reaches chainstate stores directly
- web bridge remains read-oriented except for explicitly scoped commands

Migration strategy:

- isolate explorer modules and routes
- replace direct backend lookups with query view models
- add local-only versus remote-safe endpoint classification

Test requirements:

- explorer query contract tests
- endpoint-scope tests
- load tests for read-heavy explorer queries

Rollback difficulty:

- low to medium

Why it is medium risk:

- changes user-facing query paths
- should not affect canonical state if boundaries are kept read-only

9. Introduce a canonical `StateSnapshot` model for read composition

Current problem:

- many screens and clients likely compose related information from separate read
s taken at different times
- there is no strong internal representation of “these values belong to the same
 backend view”

Target state:

- `QueryService` exposes snapshot objects containing:
  - chain revision
  - tip metadata
  - wallet summary
  - mempool summary
  - network identity
  - peer/sync status where relevant

Migration strategy:

1. define snapshot models used internally first
2. expose them on new query endpoints
3. migrate GUI screens to the new snapshot family

Test requirements:

- snapshot internal-consistency tests
- stale-snapshot versus fresh-snapshot handling tests
- revision-mismatch rejection or warning tests

Rollback difficulty:

- low to medium

Why it is medium risk:

- it changes the read contract and some view composition logic
- it materially improves consistency without touching consensus semantics

10. Create a `ConfigService` and upgrade-safe runtime configuration boundary

Current problem:

- configuration, environment interpretation, and network-specific settings likely remain spread across CLI, runtime, API, and GUI layers

Target state:

- one service owns:
  - config loading
  - config validation
  - network identity normalization
  - config revision or checksum
  - migration warnings

Migration strategy:

- centralize parsing first
- centralize validation second
- finally route switch and restart flows through the validated configuration object

Test requirements:

- invalid-config tests
- mixed-network config tests
- restart-after-config-change tests

Rollback difficulty:

- medium if all entrypoints come to depend on it

Why it is medium risk:

- configuration bugs can affect the whole runtime
- however the change remains operational rather than consensus-rule-changing

11. Extract a `RecoveryService` façade

Current problem:

- crash recovery, replay entry, and rebuild semantics are likely scattered across `Src/Main/crash_recovery.py`, consensus helpers, and store-specific rebuild methods

Target state:

- `RecoveryService` coordinates:
  - restart-time recovery decision
  - replay invocation
  - store rebuild progress reporting
  - post-recovery readiness signaling

Migration strategy:

- wrap the current recovery entrypoints first
- add structured status results and tracing
- only later consolidate internal logic

Test requirements:

- restart classification tests
- replay-required versus replay-not-required tests
- recovery-status API tests

Rollback difficulty:

- medium

Why it is medium risk:

- it affects startup semantics and readiness
- but can begin as an orchestration façade around existing behavior

12. Introduce a lightweight domain-event persistence layer for operator-critical actions

Current problem:

- important actions such as send intents, lifecycle commands, and network switches may not leave a single reliable operator-facing trail

Target state:

- persist a minimal journal of:
  - lifecycle commands
  - network switches
  - send intents
  - recovery actions

Migration strategy:

- start with append-only event records
- keep them separate from consensus-critical journals
- add query endpoints for inspection

Test requirements:

- journal append tests
- restart visibility tests
- duplicate command idempotency tests using the journal

Rollback difficulty:

- low to medium

Why it is medium risk:

- it introduces new persistence and operational behavior
- it dramatically improves production support and rollback reasoning

Medium-risk refactor conclusion

These refactors are the heart of the production strategy because they solve real architectural problems without directly rewriting consensus rules. They change ownership, interfaces, and runtime topology enough to remove ambiguity, but they can be staged behind delegation façades and strong regression tests. They are the correct middle layer between cosmetic cleanup and dangerous protocol surgery.

High-Risk / Consensus-Sensitive Refactors

1. Rework block acceptance sequencing only if evidence proves it is necessary

Why it is dangerous:

- block acceptance currently spans `Src/Main/consensus.py`, `Src/Main/blockveri.py`, `Src/Main/txveri.py`, `Src/Storage/bondstake.py`, `Src/Storage/private_notes_store.py`, `Src/Blockchain/blockchain.py`, and reorg/recovery logic
- changing sequencing can alter canonical truth, recovery semantics, or state-hash outcomes

Problems it would solve if needed:

- clearer atomic commit boundaries
- fewer partial-commit ambiguity windows
- easier reasoning about extension-state application
- easier crash-recovery proofs

Proof and tests required first:

- end-to-end acceptance trace for canonical blocks
- live apply versus replay equivalence tests
- crash-mid-commit simulation tests
- reorg-after-commit tests
- state-hash equivalence tests across restart

Preparatory refactors that should happen first:

- tracing
- repository wrappers
- explicit write transaction boundaries
- explicit extension-state hook contracts

Should it be postponed?

- yes, unless production evidence shows a real correctness problem

2. Rework reorg and replay internals only behind equivalence harnesses

Why it is dangerous:

- reorg logic is one of the deepest sources of hidden state interaction
- it touches UTXO state, private state, bond/stake state, mempool repair, and recovery semantics

Problems it would solve:

- improved determinism
- reduced complexity in rollback and replay
- better crash recovery confidence

Proof and tests required first:

- synthetic deep-reorg suite
- mixed public/private reorg suite
- payout-set rollback suite
- private note/nullifier rollback suite
- replay-from-height equivalence suite

Preparatory work first:

- invariant logging
- state snapshot comparison helpers
- reorg event tracing

Should it be postponed?

- almost certainly, until outer layers are cleaner and the equivalence suite exi
sts

3. Refactor private transaction validation only after the proof model is ful
ly pinned down

Why it is dangerous:

- this is the highest inflation-risk area besides emission and payout logic
- the prior audit already found proof-related test failures and proof-gate ambig
uity

Problems it would solve:

- cleaner proof boundary
- better transformation isolation
- stronger replay determinism

Proof and tests required first:

- every failing proof-path test fixed or deliberately retired with replacement
- malformed-proof rejection tests
- tx-body mutation invalidation tests
- verifier unavailable fail-closed tests
- block-inclusion proof revalidation tests

Preparatory work first:

- isolate proof statement construction from transport and wallet UX
- add canonical trace points for proof verification timing
- document authoritative proof-verification owner

Should it be postponed?

- yes, except for bug fixes and proof-hardening changes required to close known
gaps

4. Refactor bond/stake eligibility and settlement only with full economic re
play coverage

Why it is dangerous:

- it affects miner eligibility, payout sets, slashing, activation status, and re
ward flow
- any bug can become a chain-split or supply-accounting bug

Problems it would solve:

- clearer miner-qualification model
- better separation between canonical bond state and cached views
- easier reasoning about payout journals and replay

Proof and tests required first:

- miner-eligibility equivalence tests
- payout-set replay tests

- activation/cooldown reorg tests
- slashing and penalty replay tests if supported
- reward-accounting tests across long reorg sequences

Preparatory work first:

- read-model extraction for bond/stake dashboards
- invariant documentation
- commit-order tracing

Should it be postponed?

- yes, unless evidence shows a consensus bug or a production-unacceptable ambiguity

5. Change the atomic commit model only with a formal write-order design

Why it is dangerous:

- the current system already has an order, even if it is not perfectly documented
- changing it can create crash-recovery or replay divergence

Problems it would solve:

- stronger canonical state ownership
- fewer implicit write dependencies
- easier proof of atomicity across chainstate and side state

Proof and tests required first:

- write-order trace capture
- crash injection between every commit boundary
- restart integrity checks
- state-hash comparison after fault injection

Preparatory work first:

- repository boundary cleanup
- explicit write transactions
- commit-journal design

Should it be postponed?

- yes, until production telemetry shows it is necessary or until mainnet-readiness requires stronger guarantees than the present design can prove

6. Unify consensus authority only after outer duplication is removed

Why it is dangerous:

- premature unification often hides rather than removes complexity
- multiple validator classes may currently carry distinct responsibilities that need to remain explicit

Problems it would solve:

- cleaner mental model
- fewer duplicate rule entrypoints
- easier protocol auditing

Proof and tests required first:

- rule-by-rule ownership map
- no-op wrapper layer proving equivalent behavior
- differential validation tests between old and new paths

Preparatory work first:

- better documentation
- explicit interfaces around validators
- event and log tracing for every consensus decision point

Should it be postponed?

- yes, unless there is a proven conflicting-authority bug in the consensus core itself

7. Change miner template generation carefully and late

Why it is dangerous:

- template generation spans tx selection, fee handling, payout planning, and consensus-critical candidate structure
- private/public coexistence makes the risk higher

Problems it would solve:

- cleaner miner-service ownership
- better predictability of candidate selection
- clearer separation between miner policy and consensus rules

Proof and tests required first:

- block-template determinism tests
- fee and payout correctness tests
- private/public mixed template tests
- template versus final-accepted-block equivalence tests

Preparatory work first:

- `MinerService` boundary
- query-only miner status surfaces
- tx lifecycle normalization

Should it be postponed?

- yes, unless mining instability is materially blocking production

8. Change fork choice only if correctness is actually in doubt

Why it is dangerous:

- fork choice is one of the few places where a “cleaner design” can create a direct consensus split

Problems it would solve:

- almost none if the current deterministic chainwork rule is already correct

Proof and tests required first:

- full fork-choice specification
- adversarial competing-chain tests

- tie-break determinism tests

Should it be postponed?

- yes

9. Rework public/private conservation accounting only if invariants cannot be proven otherwise

Why it is dangerous:

- mixed-domain conservation is one of the most sensitive parts of the entire protocol
- even a subtle accounting change could create hidden inflation or false invalidation

Problems it would solve:

- cleaner mental model around public/private boundary conditions
- better auditability if the current implementation is too fragmented

Proof and tests required first:

- cross-domain conservation suite
- proof and nullifier equivalence suite
- replay and reorg conservation suite
- fee and reward interaction tests across mixed blocks

Preparatory work first:

- stronger invariant documentation
- explicit accounting trace points
- separation of private proof plumbing from conservation checks

Should it be postponed?

- yes, unless the existing model cannot be made production-provable without a targeted redesign

10. Refactor state-hash computation and verification only with a full replay oracle

Why it is dangerous:

- state hashes often sit at the edge of “looks like bookkeeping” and “actually consensus-critical”
- if computed inconsistently across live apply, replay, or role-specific paths, they can create severe drift

Problems it would solve:

- clearer canonical state-commit evidence
- stronger crash-recovery integrity checks

Proof and tests required first:

- live-versus-replay state-hash equality tests
- multi-domain state-hash coverage tests
- restart integrity tests

Preparatory work first:

- repository and commit tracing
- explicit state-hash owner definition

Should it be postponed?

- yes, unless current production evidence shows state-hash ambiguity

11. Introduce stricter single-writer enforcement inside canonical write domains only after topology cleanup

Why it is dangerous:

- aggressive runtime lock enforcement can surface previously hidden assumptions and break existing operational flows

Problems it would solve:

- stronger protection against multi-writer corruption
- clearer runtime topology guarantees

Proof and tests required first:

- storage-open topology tests
- role-to-store ownership map
- recovery and restart tests under writer lock conditions

Preparatory work first:

- supervisor ownership
- repository boundaries
- runtime topology tracing

Should it be postponed?

- yes, until the control plane is centralized enough to enforce it predictably

High-risk refactor conclusion

The high-risk work is not off the table forever. It is simply not the right starting point. The correct order is to make the system more observable, more isolated, and more testable first. Once the outer ambiguity is removed, the team can assess whether any deep consensus changes are actually required, or whether the safer outcome is to keep the current consensus core and only harden its edges.

Module Decomposition Strategy

Decomposition goals

Module decomposition should pursue five outcomes:

- smaller reasoning units
- explicit side-effect boundaries
- clear domain ownership
- narrower imports between layers
- easier test targeting

It should not pursue decomposition for its own sake. The best decomposition keeps consensus-critical flows legible without scattering them into dozens of tiny files.

Consensus decomposition

What should remain together:

- the canonical block-acceptance orchestration path
- the reorg application path
- replay/recovery logic that must stay semantically aligned with live acceptance

What it should be split into:

- `consensus_engine.py`
  - authoritative orchestration façade
  - owns block acceptance entrypoint, reorg entrypoint, replay entrypoint
- `consensus_validation.py`
  - composes `BlockVerifier` and `TransactionVerifier`
  - exposes explicit validation results and reject codes
- `consensus_commit.py`
  - chainstate and side-state commit sequencing
  - no transport or logging policy
- `consensus_recovery.py`
  - crash recovery and replay routines
- `consensus_events.py`
  - structured event emission, not business logic
- `consensus_invariants.py`
  - runtime assertions and invariant-check helpers

What should become interfaces:

- `IChainstateRepository`
- `IPrivateStateRepository`
- `IBondStakeRepository`
- `IBlockRepository`
- `IMempoolRepository`

What should remain coarse-grained:

- the state machine that advances canonical truth

The point is to make the consensus core easier to audit without fragmenting the actual authority path.

Transaction pipeline decomposition

Today the transaction pipeline spans `Src/Transactions/tx.py`, `txin.py`, `txout.py`, `txmanager.py`, `txvalidation.py`, `fee.py`, `coinbase.py`, `Src/Main/txveri.py`, `Src/Network/sendtx.py`, wallet send flows, and mempool code. That is too broad for maintainable reasoning.

Recommended decomposition:

- `tx_domain/`
  - transaction model types
  - serialization/deserialization
  - canonical amount and fee utilities
- `tx_validation/`
  - consensus-critical tx validation
  - policy-only mempool checks
  - shared error codes
- `tx_lifecycle/`
  - lifecycle records
  - intent ids
  - status transition helpers
- `tx_submission/`
  - backend-owned submission orchestration

  - relay requests
- `tx_query/`
  - read-side history and inspection views

What should remain together:

- canonical tx validity logic

What should become separate:

- lifecycle and UX-oriented send handling
- relay and networking concerns

Private transaction pipeline decomposition

This needs special care because the goal is not to over-modularize a consensus-s
ensitive area. The recommended split is:

- `private_domain/`
  - canonical note, nullifier, commitment, witness, and proof request structures
- `private_build/`
  - wallet-side construction and preview
- `private_proof/`
  - proof statement generation, prover integration, verifier integration
- `private_validation/`
  - consensus-critical validation entrypoints
- `private_state/`
  - note store, nullifier state, commitment-root management, replay helpers
- `private_query/`
  - wallet and explorer read models only

What should remain together:

- the final authoritative validation path used for block acceptance

What should be isolated behind interfaces:

- prover backend calls
- verifier backend calls
- note-state repository operations

The big goal is to keep proof plumbing, wallet UX, and canonical validation from
 bleeding into one another.

Wallet logic decomposition

Recommended modules:

- `wallet_service.py`
  - top-level owner
- `wallet_keys.py`
  - key management and secure loading
- `wallet_unlock.py`
  - unlock policy and in-memory secret lifetime
- `wallet_send_public.py`
  - public send intent creation
- `wallet_send_private.py`
  - private send intent creation
- `wallet_preview.py`
  - preview and confirm data
- `wallet_history.py`
  - query-only history composition

- `wallet_events.py`
  - event publishing

What should never live in wallet GUI code:

- canonical lifecycle state
- mempool admission results
- final transaction status decisions

API layer decomposition

Recommended modules:

- `api_gateway.py`
- `api_commands_wallet.py`
- `api_commands_node.py`
- `api_queries_chain.py`
- `api_queries_wallet.py`
- `api_queries_status.py`
- `api_events.py`
- `api_auth.py`
- `api_errors.py`
- `api_versioning.py`
- `api_rate_limit.py`

What should remain together:

- transport concerns and request/response normalization

What should move out:

- runtime construction
- business logic
- direct storage access

GUI decomposition

`Src/GUI/gui.py` should be split into:

- `gui_app.py`
  - top-level assembly only
- `gui_store.py`
  - authoritative frontend state store
- `gui_api_client.py`
  - HTTP and event transport
- `gui_events.py`
  - backend-event adapters
- `gui_views_wallet.py`
- `gui_views_node.py`
- `gui_views_explorer.py`
- `gui_views_settings.py`
- `gui_controllers_wallet.py`
- `gui_controllers_node.py`
- `gui_controllers_navigation.py`
- `gui_models.py`
  - frontend view models only
- `gui_errors.py`

What should remain together:

- the GUI assembly root

What should move out:

- widget-local business logic
- retry guesswork
- node orchestration

Node launcher and runtime decomposition

Recommended modules:

- `supervisor_service.py`
  - process lifecycle owner
- `runtime_topology.py`
  - role definitions and dependency graph
- `runtime_health.py`
  - liveness/readiness semantics
- `runtime_switch.py`
  - network switch orchestration
- `runtime_shutdown.py`
  - ordered shutdown
- `runtime_process_registry.py`
  - process handles and PID state

Existing entrypoints such as `Src/Main/runnode.py`, `runminer.py`, `rungui.py`, and `stop.py` should become thin shells over these modules rather than independent behavior centers.

Storage layer decomposition

Recommended modules:

- `repositories/block_repository.py`
- `repositories/header_repository.py`
- `repositories/utxo_repository.py`
- `repositories/mempool_repository.py`
- `repositories/peer_repository.py`
- `repositories/private_state_repository.py`
- `repositories/bondstake_repository.py`
- `repositories/recovery_repository.py`
- `repositories/query_views.py`
- `storage_transactions.py`
- `storage_schema.py`
- `storage_migrations.py`

What should remain together:

- the write-transaction discipline for consensus-critical state

What should move out:

- ad hoc direct LMDB access from higher layers

P2P decomposition

Recommended modules:

- `peer_manager.py`
- `sync_service.py`
- `p2p_transport.py`
- `message_handlers.py`
- `peer_scoring.py`
- `ban_manager.py`

- `block_download.py`
- `tx_relay.py`

What should remain together:

- transport and message parsing

What should be separate:

- consensus validation
- peer-state persistence
- relay policy

Mining decomposition

Recommended modules:

- `miner_service.py`
- `template_builder.py`
- `candidate_selection.py`
- `reward_planner.py`
- `pow_runner.py`
- `gpu_runner.py`
- `miner_status.py`

What should remain together:

- template and candidate assembly

What should be separate:

- PoW execution backends
- status reporting
- policy configuration

Current-to-target module migration map

The safest decomposition is not to invent an entirely new tree at once. It is to map existing files into target ownership domains and migrate them incrementally.

Recommended mapping:

Current module	Immediate target bucket	Long-term owner
`Src/Main/consensus.py`	`consensus_engine.py`, `consensus_commit.py`, `consensus_recovery.py` wrappers	`ConsensusEngine`
`Src/Main/blockveri.py`	`consensus_validation.py`	`ConsensusEngine`
`Src/Main/txveri.py`	`tx_validation/consensus.py`	`ConsensusEngine` and `MempoolService` for policy adapters
`Src/Transactions/txmanager.py`	`tx_submission/` and `wallet_send_public.py` split points	`WalletService` and `MempoolService`
`Src/Transactions/private_layer.py`	`private_build/`, `private_proof/`, `private_validation/` split points	`WalletService` and `ConsensusEngine`
`Src/Storage/private_notes_store.py`	`private_state_repository.py` plus `PrivateStateManager` façade	`PrivateStateManager`
`Src/Storage/bondstake.py`	`bondstake_repository.py` plus `BondStakeManager` façade	`BondStakeManager`
`Src/Storage/mempool.py`	`mempool_repository.py` plus `MempoolService` façade	`MempoolService`
`Src/Storage/peerstore.py`	`peer_repository.py` plus `PeerManager` façade	`PeerManager`

`Src/Api/apis.py`	`api_gateway.py`, command/query modules	`APIGateway`
`Src/Api/auth.py`	`api_auth.py` and auth middleware	`APIGateway`
`Src/GUI/gui.py`	GUI store/client/controller/view modules	`GUIStateStore` plus GUI layers
`Src/Main/runnode.py`	thin CLI shell over `SupervisorService` and runtime modules	`SupervisorService`
`Src/Main/stop.py`	thin CLI shell over `SupervisorService`	`SupervisorService`
`Src/Network/sync.py`	`sync_service.py`	`SyncService`
`Src/Network/peers.py`	`peer_manager.py` and `peer_scoring.py`	`PeerManager`
`Src/Miner/miner.py`	`miner_service.py`, `template_builder.py`, `pow_runner.py`	`MinerService`

This mapping matters because it turns refactoring into a series of ownership moves instead of a vague cleanup campaign.

Dependency rules for the target module graph

The target codebase should follow strict dependency rules:

1. GUI modules may depend on API client and view models only.
2. API modules may depend on service interfaces only.
3. Query modules may depend on repository read interfaces and read-only service interfaces.
4. Service modules may depend on repositories, domain modules, and selected peer services through interfaces.
5. Repository modules may depend on storage utilities only.
6. Domain modules should avoid depending on transport or UI code entirely.

Forbidden dependency examples:

- GUI importing repository helpers
- API route importing `Src/Main/consensus.py` or low-level store classes directly
- miner code importing GUI state or API transport
- query code importing write-oriented runtime assembly code
- wallet GUI code importing transaction verifier internals

These dependency rules should be enforced by review policy and, if feasible, import-lint tooling.

Shared-library versus domain-module rules

One of the easiest ways to create new coupling is to dump unrelated logic into a shared utility package. The codebase should distinguish carefully between:

- shared utilities
- domain libraries
- orchestration modules

Shared utilities may contain:

- pure serialization helpers
- generic error primitives
- configuration parsing primitives
- logging helpers

They should not contain:

- hidden service locators
- global runtime state

- convenience functions that cross domain boundaries
- implicit mutations against stores or processes

Domain libraries should contain only the logic intrinsic to their domain:

- tx types
- fee math
- note structures
- peer score calculations
- lifecycle state enums

Orchestration modules should coordinate domains but not quietly become new sourc
es of truth.

Decomposition acceptance criteria

A module split should not be considered successful merely because the file got s
maller. It should meet at least three of these criteria:

- fewer reasons for the module to change
- clearer import boundaries
- better test locality
- reduced direct side effects
- clearer ownership of mutations
- easier runtime tracing

If a split increases indirection without improving those properties, it is not w
orth keeping.

Decomposition strategy conclusion

The module strategy is intentionally uneven. Consensus remains coarse-grained re
lative to GUI or API, while product and control-plane code is split more aggress
ively. That is deliberate. The codebase needs smaller modules where coupling is
accidental, and larger guarded modules where coupling is intrinsic to canonical
truth.

Service Boundary and Ownership Strategy

Service-boundary goals

The long-term architecture should make three things obvious:

- who owns mutation
- who exposes read models
- who controls runtime

If a maintainer cannot answer those questions from the service map alone, the ar
chitecture is still too ambiguous.

Core service table

| Service | Owns | Must never own | Allowed mutations | Exposes | May call | Har
d boundary |
|---|---|---|---|---|---|---|
| `ConsensusEngine` | canonical block acceptance, reorg, replay, validity orches
tration | GUI state, HTTP, process lifecycle | chainstate advancement, extension
-state advancement through repositories | block-accept results, consensus events
, invariant status | repositories, `MempoolService`, `EventBus` | no direct GUI/
API/bootstrap logic |
| `ChainstateManager` | tip, block index, UTXO truth, state revision | wallet ke
ys, peer policy, GUI state | canonical chainstate writes | chain snapshots, revi

sion ids | repositories | no transport or policy UI code |
`PrivateStateManager`	commitments, nullifiers, note-root progression, privat e replay support	wallet UX, proof-generation UX	canonical private-state writ es	private-state revision info, validation hooks	repositories, `ConsensusEng ine`	no API or GUI ownership
`BondStakeManager`	canonical bond/stake truth, miner eligibility state, payo ut journals	GUI dashboards, process lifecycle	bond/stake writes via consensu s hooks	miner eligibility snapshots, payout state summaries	repositories, `C onsensusEngine`	no direct wallet UX ownership
`MempoolService`	tx admission, mempool truth, eviction, relay-ready queue st ate	final transaction confirmation truth, GUI retries	mempool insert/update/ evict	mempool snapshots, admission results, tx status hints	`ConsensusEngine `, `PeerManager`, repositories	no wallet-key logic
`MinerService`	candidate building, miner-run state, template assembly	cano nical block acceptance, lifecycle of unrelated roles	template and mining-run m utations only	mining status, candidate summaries	`MempoolService`, `BondStak eManager`, `NodeService`	miner policy must not define consensus validity
`PeerManager`	peer records, scoring, bans, selection	block validity, walle t state	peer-state writes	peer lists, peer health, sync candidates	reposit ories, `SyncService`	cannot decide consensus truth
`SyncService`	sync workflow, download scheduling, catch-up state	canonical validation rules, wallet state	sync session state	sync progress, backlog st atus	`PeerManager`, `ConsensusEngine`	must submit blocks to consensus, not a ccept them itself
`WalletService`	keys, unlock state, send intents, wallet command flows	blo ck validity, mempool truth, process lifecycle	wallet-intent and wallet-local s tate mutations	previews, histories, lifecycle views	`NodeService`, `MempoolS ervice`, `QueryService`, `EventBus`	GUI must not bypass it for wallet mutation s
`NodeService`	backend runtime commands, node-scoped operations, node status model	process spawning policy, GUI state	node runtime commands through super visor	status views, command results	`SupervisorService`, `ConsensusEngine`, `MempoolService`, `QueryService`	should not instantiate arbitrary new consensu s graphs
`SupervisorService`	process lifecycle, startup ordering, shutdown ordering, restart policy, readiness aggregation	chainstate truth, wallet state, explorer queries	process start/stop/restart/switch operations	lifecycle status and h ealth	runtime modules, `NodeService`, `EventBus`	only lifecycle authority
`APIGateway`	HTTP transport, auth, validation, versioning, rate limits	bus iness truth, canonical state mutation ownership	none except transport/session concerns	command/query responses, event streaming	all backend services throu gh interfaces	must not bootstrap canonical runtimes
`QueryService`	read-only projections and composed views	canonical writes, runtime control	cache refresh only	balance/history/tip/peer/query views	re positories, read interfaces on services	never mutate canonical truth
`EventBus`	typed backend event distribution	canonical decision-making	ev ent publication buffers	event stream	all services publish or subscribe	eve nts inform; they do not define truth
`GUIStateStore`	frontend displayed state, revision-aware local cache, screen models	canonical blockchain state, wallet keys, lifecycle authority	fronten d store mutations only	view models for screens	`APIGateway`, GUI event adapt ers	client only

Ownership rules by domain

Chain truth

- `ConsensusEngine` and `ChainstateManager` are the only legitimate owners
- API may expose
- GUI may display
- no other layer may reinterpret canonical truth into an alternative source of a uthority

Private consensus state

- `PrivateStateManager` owns the canonical state transitions
- wallet code may construct candidate private transactions
- wallet code must never decide whether private state advanced canonically

Bond/stake and miner eligibility

- `BondStakeManager` owns canonical bond/stake transitions
- `MinerService` consumes eligibility snapshots
- miner code must not self-certify eligibility

Mempool truth

- `MempoolService` owns transaction admission, eviction, and local mempool state
- wallet and GUI only receive admission outcomes

Wallet intent and unlock state

- `WalletService` owns both
- GUI must never hold authoritative unlock state
- API must never implement wallet business logic directly

Runtime lifecycle

- `SupervisorService` owns start, stop, restart, readiness, and network switch orchestration
- `NodeService` exposes runtime commands through that owner
- GUI is a client

Read models

- `QueryService` owns all composed views intended for the GUI or explorer
- direct reads from repositories should be reserved for service internals and tests

Boundaries that should never be crossed

- GUI must not talk to LMDB-backed repositories directly
- GUI must not control node roles by shelling around the supervisor surface
- API route handlers must not instantiate consensus-core objects on the fly
- miner code must not bypass consensus validation
- peer code must not accept blocks directly into canonical state
- query code must not mutate mempool, chainstate, or wallet truth
- wallet code must not infer transaction confirmation from polling heuristics when backend lifecycle state exists

Dependency direction

The dependency flow should look like this:

1. repositories and domain modules at the bottom
2. consensus, mempool, bond/stake, private state, peer, sync, miner, wallet services above them
3. query service and event bus reading from authoritative services and repositories
4. API gateway as transport over services
5. GUI state store and views at the top

Anything that inverts that flow should be treated as suspect.

Synchronous versus asynchronous boundaries

Use synchronous boundaries for:

- command validation and immediate command acknowledgment
- block acceptance and transaction admission
- lifecycle commands where a definitive accepted/rejected result is needed

Use asynchronous boundaries for:

- event propagation
- long-running sync status
- mining telemetry
- confirmation updates
- replay/recovery progress

This split reduces the temptation to overload command endpoints with pseudo-streaming behavior.

Allowed service-call graph

The service graph should be narrow and directional.

Allowed examples:

- `APIGateway -> WalletService`
- `APIGateway -> NodeService`
- `APIGateway -> QueryService`
- `APIGateway -> EventBus` transport adapter only
- `WalletService -> MempoolService`
- `WalletService -> QueryService`
- `NodeService -> SupervisorService`
- `NodeService -> QueryService`
- `MinerService -> MempoolService`
- `MinerService -> BondStakeManager`
- `SyncService -> PeerManager`
- `SyncService -> ConsensusEngine`
- `ConsensusEngine -> ChainstateManager`
- `ConsensusEngine -> PrivateStateManager`
- `ConsensusEngine -> BondStakeManager`
- `QueryService -> repositories and read-only service interfaces`

Disallowed examples:

- `GUIStateStore -> repositories`
- `APIGateway -> ConsensusEngine` by constructing it locally
- `QueryService -> ConsensusEngine` for ad hoc writes
- `PeerManager -> ConsensusEngine` to decide block validity
- `MinerService -> ChainstateManager` to self-commit candidate results
- `WalletService -> SupervisorService` for arbitrary lifecycle mutations

This call graph is what prevents “temporary” shortcuts from becoming permanent architecture damage.

Mutation routing examples

Example: send transaction

Desired route:

`GUI -> APIGateway -> WalletService -> MempoolService -> EventBus`

Supporting reads:

`GUI <- APIGateway <- QueryService`

What must not happen:

- GUI directly probing several endpoints and inventing a tx state
- API route directly mutating mempool and wallet state in one ad hoc function
- wallet helper directly broadcasting on the network without going through mempool ownership

Example: start node

Desired route:

`GUI or CLI -> APIGateway/CLI shell -> NodeService -> SupervisorService -> runtime processes`

What must not happen:

- GUI shelling directly to `runnode.py`
- API deciding readiness without supervisor evidence
- two different services trying to own restart policy

Example: block acceptance

Desired route:

`P2P/Sync -> ConsensusEngine -> ChainstateManager/PrivateStateManager/BondStakeManager -> EventBus`

What must not happen:

- peer code accepting blocks directly into canonical stores
- miner code bypassing consensus and self-committing
- query code triggering recovery or replay

Service boundary anti-patterns to remove

- façade classes that still expose underlying store handles
- services that return raw LMDB transactions or mutable internals
- API wrappers that smuggle business logic back into transport
- query services that silently refresh caches by triggering writes in authoritative services
- event handlers that perform hidden domain mutations without an explicit command path

These anti-patterns matter because they recreate the same ambiguity under new names.

Boundary enforcement mechanisms

The architecture should not rely only on discipline. Use enforcement mechanisms where possible:

- import-layer linting
- runtime assertions in debug builds
- test cases that verify forbidden call paths are absent
- thin service interfaces that do not expose internals
- code review checklists that name ownership and boundary impact explicitly

That is how the codebase stays clean after the initial refactor wave rather than

slowly regressing.

Boundary strategy conclusion

The long-term production architecture should not be “more services everywhere.” It should be clearer authority everywhere. Some of those authorities become services because that is the clearest way to isolate them. Others, especially the consensus core, should remain tightly controlled internal domains behind a single public owner.

Refactoring the Control Plane

Control-plane diagnosis

The current control-plane problem is not simply that process management is messy. It is that operational authority is distributed across too many layers:

- CLI entrypoints in `Src/Main/runnode.py`, `Src/Main/runminer.py`, and `Src/Main/stop.py`
- role-specific runtime modules such as `Src/Node/walletnode.py` and `Src/Node/minernode.py`
- API bootstrap and auth/control surfaces in `Src/Api/apis.py` and `Src/Api/auth.py`
- GUI fallback behavior in `Src/GUI/gui.py`
- network-switch and config paths that likely reach across several of those layers

That means the system still allows more than one component to behave like an orchestrator. Even when the code works, that is a production liability. It creates ambiguity around:

- who is allowed to start a role
- who is allowed to stop a role
- who decides readiness
- who interprets a half-initialized backend
- who is responsible for restart after failure
- who coordinates a network switch across local processes and persisted state

The result is avoidable fragility. A production-grade control plane should make lifecycle authority explicit and narrow.

Target control-plane architecture

The target model should be:

- `SupervisorService` is the only lifecycle authority
- `NodeService` is the domain façade for node-level commands and status
- `APIGateway` exposes lifecycle commands and status queries but does not perform orchestration itself
- GUI only invokes commands and renders status
- role entrypoints become thin adapters that register with or are launched by the supervisor

The basic topology should be:

```text
GUI / CLI
  -> APIGateway
    -> NodeService
      -> SupervisorService
        -> ProcessRegistry
        -> StartupPlanner
```

```
        -> ReadinessAggregator
        -> ShutdownCoordinator
        -> NetworkSwitchCoordinator
          -> role processes
```

The most important rule is that the control plane is responsible for process truth, not consensus truth. It does not validate blocks or transactions. It starts, stops, monitors, and sequences the components that do.

Lifecycle ownership model

`SupervisorService` should own:

- process spawn and PID tracking
- startup ordering
- readiness aggregation
- health checks
- restart policy
- crash classification
- shutdown ordering
- network switch orchestration
- operator-visible lifecycle events

`NodeService` should own:

- semantic commands such as `start_full_node`, `stop_miner`, `restart_backend`, `switch_network`
- node-scoped status models
- translation between operator intent and supervisor operations

`APIGateway` should own:

- request validation
- auth checks
- idempotency keys for lifecycle commands
- response normalization

The GUI should own:

- interaction state
- display of progress, readiness, and failures

The GUI should not own:

- process orchestration decisions
- hidden retries that change backend state
- direct shelling into role entrypoints
- self-declared readiness logic

Failure-containment goals for the control plane

The supervisor redesign should be judged partly by how well it contains failures.

Required containment properties:

- API crash does not change canonical state and does not require node restart
- GUI crash does not affect backend state at all
- miner crash does not corrupt node readiness or wallet usability
- failed health check does not trigger process flapping without bounded policy
- partial startup in one role does not present full readiness to clients

- network-switch failure does not leave old and new network roles active simultaneously

These goals should become explicit supervisor invariants.

Supervisor operating rules

The supervisor should follow several non-negotiable rules:

- it must never infer consensus validity from liveness
- it must never bypass service interfaces to mutate domain state directly
- it must serialize lifecycle operations that target the same role or storage root
- it must reject conflicting operations rather than attempting clever implicit merges
- it must publish enough status for operators and the GUI to understand intermediate states

Those rules keep the control plane from becoming a new source of hidden side effects.

Startup ordering

One of the most valuable control-plane refactors is to make startup ordering explicit.

Recommended order:

1. config resolution and lock acquisition
2. storage compatibility checks
3. role plan construction
4. process spawn
5. local transport bind
6. internal dependency checks
7. readiness proof publication
8. API exposure of ready state
9. GUI unblocks operational controls

Why this matters:

- liveness is not readiness
- a process that bound a port is not necessarily safe to serve commands
- a node that loaded some stores is not necessarily safe to accept wallet commands
- an API that can answer `/status` is not necessarily safe to answer `/send`

The supervisor should expose three distinct states per role:

- `starting`
- `live`
- `ready`

And optionally:

- `degraded`
- `recovering`
- `stopping`
- `stopped`
- `failed`

That distinction eliminates a large class of product bugs where the frontend or operator mistakes partial initialization for usable state.

Readiness checks versus health checks

Today the system likely mixes these concepts.

Health checks answer:

- is the process alive
- is the event loop still responding
- is the port bound

Readiness checks answer:

- did the process load its required stores
- did it validate schema and network identity
- did it complete recovery
- did it wire required internal services
- is it safe to accept commands

Production-grade control planes never collapse those into one bit.

Recommended readiness evidence:

- network identity loaded and consistent
- chainstate opened successfully
- recovery complete or explicitly skipped
- consensus service entered stable state
- wallet service opened without incompatible schema
- mempool service initialized
- peer manager available if role requires it

Recommended health evidence:

- process heartbeat
- request loop responsiveness
- memory pressure warning state
- internal error-rate threshold

Current versus target runtime topology

Current topology likely behaves roughly like this:

```text
GUI
  -> API routes
  -> fallback local control logic
CLI entrypoints
  -> runnode.py / stop.py / runminer.py
API bootstrap
  -> backend assembly
role modules
  -> node/miner/wallet runtimes
shared stores
  -> LMDB-backed state
```

The danger in that shape is not that it cannot run. The danger is that too many places can believe they are the operational entrypoint.

Target topology should look like:

```text
```

```
GUI / CLI
  -> APIGateway or thin CLI shell
    -> NodeService
      -> SupervisorService
        -> ProcessRegistry
        -> StartupPlanner
        -> ReadinessAggregator
        -> ShutdownCoordinator
        -> NetworkSwitchCoordinator
          -> role processes
            -> backend services
              -> repositories
```

This topology is easier to reason about because lifecycle truth is centralized even if domain truth remains distributed across backend services.

Lifecycle command state machine

Every lifecycle command should move through explicit states:

1. requested
2. validated
3. accepted
4. executing
5. awaiting readiness or shutdown proof
6. completed or failed

For example, `start_node` should not jump straight from request to “done” because a process spawned. It should complete only after supervisor readiness evidence says the target role is ready.

Likewise, `switch_network` should not complete merely because config changed. It should complete only after:

- roles were quiesced
- target config validated
- roles restarted on target network
- readiness succeeded under target network identity

This state machine should be visible via logs, status queries, and event stream.

Restart behavior

Restart handling should be explicit, not emergent.

Recommended restart categories:

- operator-requested restart
- crash restart
- restart after configuration change
- restart after network switch
- restart after upgrade or migration

Each category should have different rules. For example:

- crash restart may require recovery checks and degraded mode
- network switch restart must prove old-network state is no longer live
- configuration restart may preserve runtime caches if safe
- upgrade restart must run schema checks first

`SupervisorService` should persist a minimal restart journal containing:

- requested action
- target roles
- network identity
- timestamp
- pre-stop status
- restart reason
- final outcome

That journal is critical for diagnosing repeated crash loops and ambiguous opera
tor reports.

Network switching

Network switching deserves special treatment because it is one of the easiest pl
aces to create mixed state.

A safe switch should require:

1. explicit supervisor lock
2. stop or quiesce relevant processes
3. flush and close active handles
4. validate target network config
5. verify storage roots or namespaces for target network
6. clear or invalidate incompatible read caches
7. restart in target network
8. publish a `network_switched` event only after readiness

What should never happen:

- GUI modifies config and assumes the backend will converge
- API toggles a network field while active processes are still using old state
- partial switch where some services see one network and others see another

This is a place where explicit supervisor ownership pays for itself immediately.

Shutdown ordering

Shutdown should be deterministic and staged:

1. stop accepting new commands
2. notify subscribers that shutdown is beginning
3. quiesce miner or background workers
4. stop new P2P intake
5. finish or abort in-flight operations according to policy
6. flush critical journals and lifecycle state
7. close storage handles
8. mark role stopped

This matters because many “mysterious restart bugs” are actually shutdown-order
bugs that leave stores or caches in an ambiguous state.

Process isolation and shared-state coordination

Production-grade runtime discipline requires an answer to the single-writer ques
tion. The supervisor must understand which processes are allowed to open which s
tores for write, and whether any storage roots may be shared concurrently.

Recommended rule:

- consensus-critical write domains must have one writer per network instance
- read-only query processes may attach only if repository contracts explicitly s

upport it
- no process should open consensus-critical stores for opportunistic writes outside the node authority path

The supervisor should therefore maintain:

- role-to-storage ownership metadata
- lock acquisition and release rules
- startup refusal if another writer already owns the target storage root

This is not an optimization. It is a correctness boundary.

Safe migration path

Control-plane refactoring should proceed in four steps:

1. create a `SupervisorService` façade that delegates to current start/stop logic
2. route GUI and API lifecycle commands through it
3. move readiness and health aggregation into the façade
4. progressively remove direct lifecycle behavior from GUI, API bootstrap, and scattered utilities

Only after that should the internal process-management code be simplified.

Control-plane refactor conclusion

The control plane is one of the best early targets because it is currently a high-ambiguity area with high production value and only medium protocol risk. If lifecycle authority becomes explicit, many other refactors become safer automatically: API becomes cleaner, GUI becomes simpler, and the probability of multi-writer or mixed-network runtime errors drops sharply.

Refactoring the GUI for Maintainability

GUI diagnosis

`Src/GUI/gui.py` is a classic warning sign: a large file that appears to carry view logic, transport logic, local state management, lifecycle interactions, wallet flows, explorer behavior, and retry or failover decisions. Even if the GUI is functionally rich, this shape becomes a maintenance liability because:

- a UI tweak can accidentally affect backend calling behavior
- backend changes require large GUI edits in unrelated places
- state bugs become hard to isolate
- tests are hard to write at a useful granularity

The problem is not that the GUI exists. The problem is that the GUI still appears to act partly like a controller for the whole system rather than a client over stable backend contracts.

GUI target architecture

The target frontend should have five clear layers:

1. view components
2. controller logic
3. transport clients
4. one GUI state store
5. event adapters

Recommended layout:

```text
gui_app.py
  -> gui_store.py
    -> gui_models.py
    -> gui_events.py
    -> gui_api_client.py
  -> gui_views_wallet.py
  -> gui_views_node.py
  -> gui_views_explorer.py
  -> gui_views_settings.py
  -> gui_controllers_wallet.py
  -> gui_controllers_node.py
  -> gui_controllers_navigation.py
```

What each GUI module should own

`gui_store.py`:

- the authoritative frontend state tree
- last-known backend revisions
- screen-specific derived view models
- command in-flight state
- event reconciliation logic

`gui_api_client.py`:

- HTTP and streaming transport
- request headers
- request ids
- timeout and retry policy
- translation of backend envelopes into internal types

`gui_events.py`:

- subscription to backend event stream
- reconnection logic
- event ordering buffers if needed
- reconciliation triggers into the store

`gui_controllers_wallet.py`:

- wallet command intent from user actions
- calls into `gui_store` or `gui_api_client`
- no canonical business logic

`gui_controllers_node.py`:

- node start/stop/restart/switch commands
- presentation-level gating of dangerous actions

`gui_views_*`:

- rendering only
- minimal local ephemeral widget state

What the GUI must not own

- consensus truth
- wallet unlock truth
- send lifecycle truth

- mempool truth
- node readiness truth
- network identity truth
- transaction confirmation heuristics
- direct storage access
- direct shell orchestration

The GUI may cache and display. It may not redefine authority.

Low-risk GUI refactors

The safest first steps are file and dependency cleanup:

- split `gui.py` without changing behavior
- create a shared API client
- create a shared error mapper
- centralize state mutations in one store
- replace widget-local repeated request logic with store actions
- isolate explorer views from wallet and node-control views

These changes improve maintainability quickly and mostly preserve behavior.

Medium-risk GUI refactors

The next tier of changes should alter behavior in controlled ways:

- replace scattered polling with store-managed polling
- introduce backend-event consumption
- move send lifecycle rendering to backend-owned states
- move node-status interpretation to revision-aware query results
- remove GUI-side fallback orchestration logic

These are medium risk because they can change timing, race behavior, and user-visible responsiveness, but they are still far safer than deep consensus changes.

GUI state model

The store should treat every backend-facing domain as a distinct slice:

- `nodeStatus`
- `walletState`
- `walletHistory`
- `walletSendIntents`
- `chainSummary`
- `mempoolSummary`
- `peerSummary`
- `networkState`
- `settingsState`
- `uiTransient`

Each slice should include:

- `last_revision`
- `last_updated_at`
- `loading_state`
- `error_state`
- `staleness_reason` if applicable

This structure gives the GUI a deterministic way to say:

- “I have data”
- “I have stale data”

- “I have no trustworthy data”

instead of implicitly assuming freshness.

Backend event consumption model

The GUI should move toward:

- subscribe to event stream
- update store slices optimistically only when safe
- reconcile with query endpoints by revision

Example flow:

1. backend emits `transaction_intent_updated`
2. GUI store updates the intent slice if the event revision is newer
3. if the event indicates a structural change, the store triggers a query refres
h
4. views rerender from the store only

This model is much safer than each widget polling its own endpoint and inferring
 the overall system state from partial answers.

GUI wallet-flow cleanup

Wallet send flow should be divided into:

- input collection
- preview request
- confirmation action
- backend intent submission
- lifecycle tracking

The GUI should no longer:

- guess whether relay succeeded by looking for tx ids in unrelated queries
- guess whether the backend is ready based on one endpoint returning 200
- guess whether the wallet is unlocked based on error-string matching

All of those should come from normalized backend contracts.

Explorer-bridge isolation

Explorer or embedded web views should be isolated from the main GUI store so tha
t:

- read-heavy explorer interactions do not influence wallet or node-control slice
s
- any web-facing or browser-like behavior remains behind a small adapter boundar
y
- public-safe endpoints can be distinguished from privileged local ones

This also makes it easier to harden explorer behavior later without destabilizin
g core wallet screens.

GUI test strategy

Recommended test layers:

- view-model tests for `gui_store`
- controller tests for wallet and node commands
- transport contract tests with mocked backend envelopes

- event ordering and reconciliation tests
- stale-read handling tests
- error taxonomy rendering tests

The most important GUI tests are not visual. They are state-transition tests that prove the GUI stops inventing backend truth.

What should wait

The following GUI work should wait until the core store and transport boundaries are established:

- major visual redesigns
- animation and presentation refactors
- heavy client-side caching optimizations
- broad navigation rewrites

Those changes may be attractive, but they do not address the current structural risk.

GUI refactor conclusion

The GUI should be one of the earliest substantial refactors because it is high value and comparatively safe if approached as decomposition plus boundary enforcement. The goal is not a prettier GUI architecture diagram. The goal is to stop the frontend from acting like a partial orchestrator and make it a deterministic client over backend-owned truth.

Refactoring the API for Maintainability and Stability

API diagnosis

The API layer currently appears to do too many jobs:

- transport
- runtime assembly
- business logic dispatch
- local readiness interpretation
- auth and key handling
- possibly some direct storage or component access

This makes it hard to answer simple production questions such as:

- is the API merely exposing services, or is it constructing them
- is the API safe to restart independently
- does a route return authoritative truth or ad hoc assembled state
- does a command represent a domain action or a convenience wrapper over local scripts

Production APIs should be boring. They should not surprise maintainers with hidden authority.

Target API model

`APIGateway` should be a façade over real backend services.

It should do:

- auth and scope checks
- request validation
- idempotency handling for commands
- rate limiting and overload protection

- response normalization
- API versioning
- event streaming transport

It should not do:

- instantiate consensus-core components
- own wallet business logic
- own node lifecycle logic
- compose ad hoc inconsistent read responses
- interpret backend readiness from weak hints

Command/query separation

The API should split endpoints into:

- command endpoints
- query endpoints
- event stream endpoints

Command endpoints:

- mutate or request mutation
- require idempotency handling
- return accepted/rejected plus resulting lifecycle handle

Query endpoints:

- are read-only
- always return revision metadata
- may use read models or caches
- must expose freshness information

Event stream endpoints:

- deliver typed events
- include event ids and revisions
- never replace query endpoints as the source of authoritative snapshots

This separation clarifies behavior for both clients and maintainers.

Response normalization

Every API response should share a standard envelope:

- `request_id`
- `ok`
- `error`
- `state_revision`
- `tip_height`
- `tip_hash`
- `network`
- `node_role`
- `timestamp`
- endpoint-specific `data`

Command responses should additionally include:

- `accepted`
- `command_id`
- `lifecycle_id` if applicable
- `warnings`

Error responses should standardize:

- `code`
- `message`
- `retryable`
- `category`
- `details` only where safe

This structure reduces fragile client logic immediately.

Idempotent command handling

Certain commands must be safe to retry:

- start node
- stop node
- restart backend
- switch network
- unlock wallet
- create send intent
- submit built transaction

The API should accept an idempotency key or command id for these operations. The backend should persist enough command metadata to determine whether:

- the command was already applied
- it is still in progress
- it failed definitively
- it is safe to retry

This is particularly important because GUI and operator tooling will inevitably retry on timeouts.

Endpoint grouping

Recommended groups:

- `/v1/node/*`
- `/v1/wallet/*`
- `/v1/query/*`
- `/v1/admin/*`
- `/v1/events/*`
- `/v1/health/*`

Within each group:

- commands use verbs only where necessary
- queries use stable nouns and view models
- admin-only endpoints are clearly scoped

This helps prevent dangerous local-only operations from blending into general-purpose query surfaces.

Auth scope cleanup

Auth should distinguish at minimum:

- local-read
- local-write
- wallet-write
- admin-lifecycle

- debug/trace

Why this matters:

- not every local consumer should be able to restart the node
- not every wallet caller should be able to switch networks
- debug and trace endpoints can expose sensitive information if left unscoped

This matters even on localhost because untrusted local software and browser-adjacent flows are still real threat models.

Rate limiting and overload behavior

The API should defend backend stability by:

- rate limiting expensive queries
- enforcing concurrency caps for certain commands
- rejecting or degrading low-priority queries during heavy sync or recovery
- protecting event streams and explorer queries from starving command handling

This is part of production hardening, not just security hardening.

Versioning strategy

The API contract should become explicitly versioned before major refactors land.

Rules:

- add new fields compatibly
- do not silently change meanings of existing fields
- deprecate old endpoints on a schedule
- version event payloads if their schema can evolve

This is especially important because the GUI and API may not always update in lockstep forever.

Safe migration path

1. introduce response envelopes on new or wrapped endpoints
2. add command/query grouping without deleting old routes immediately
3. move route business logic into services
4. add idempotency and error taxonomy
5. add event stream transport
6. deprecate old ad hoc endpoints

That order is safe because clients can migrate gradually.

API refactor conclusion

The API is a high-value early refactor because it sits between backend authority and client confusion. Once it becomes a true gateway over services rather than a mixed runtime layer, the rest of the system becomes easier to reason about and much easier to test.

Recommended endpoint contract examples

Example command request:

```json
{
  "request_id": "client-generated-or-empty",
  "idempotency_key": "uuid-or-stable-command-key",
```

```
  "data": {
    "network": "mainnet"
  }
}
```

Example command response:

```json
{
  "request_id": "req-123",
  "ok": true,
  "accepted": true,
  "command_id": "cmd-456",
  "lifecycle_id": "lifecycle-789",
  "state_revision": 102244,
  "tip_height": 88321,
  "tip_hash": "0000...",
  "network": "mainnet",
  "node_role": "full",
  "timestamp": "2026-04-14T12:00:00Z",
  "warnings": [],
  "data": {
    "status": "starting"
  }
}
```

Example query response:

```json
{
  "request_id": "req-124",
  "ok": true,
  "state_revision": 102245,
  "tip_height": 88322,
  "tip_hash": "0000...",
  "network": "mainnet",
  "node_role": "full",
  "timestamp": "2026-04-14T12:00:01Z",
  "data": {
    "wallet_balance": "...",
    "history_revision": 88190
  }
}
```

The specific schema can differ, but the contract needs this level of predictability.

API overload and degradation policy

The gateway should be explicit about how it behaves under load.

Recommended policy:

- command endpoints retain priority over exploratory read traffic
- expensive query endpoints may shed load with retryable errors
- event-stream fan-out should have bounded buffers
- debug and trace endpoints may disable themselves or require stronger scopes under pressure

The purpose is to avoid a situation where a heavy GUI refresh or explorer query storm degrades the exact lifecycle or wallet commands the operator needs most.

Storage and Repository Refactoring Strategy

Storage diagnosis

The storage layer is one of the most dangerous areas to change casually and one of the most important areas to clean up. The prior audits already established the main problem:

- direct or loosely wrapped LMDB-backed state appears across several domains
- there are multiple runtime roles and local services that may open overlapping stores
- not all read and write paths are cleanly separated

That means the correct storage strategy is not “replace LMDB” or “rewrite persistence.” The correct strategy is to make storage ownership explicit, reduce call-site sprawl, and prove single-writer boundaries.

Repository-first strategy

The first storage refactor should be repository extraction, not schema redesign.

Every major store should have a repository interface:

- block repository
- header repository
- UTXO repository
- mempool repository
- peer repository
- private-state repository
- bond/stake repository
- recovery/replay repository

Those repositories should separate:

- canonical writes
- read-only snapshot reads
- maintenance operations
- debug or invariant inspection operations

This immediately reduces the amount of higher-level code that knows storage details.

Single-writer boundaries

The storage model should declare, in code and at runtime, which service is the only allowed writer for each domain.

Recommended ownership:

- block/header/chain tip: `ConsensusEngine` through `ChainstateManager`
- UTXO: `ConsensusEngine` through `ChainstateManager`
- private notes/nullifiers: `ConsensusEngine` through `PrivateStateManager`
- bond/stake and payouts: `ConsensusEngine` through `BondStakeManager`
- mempool: `MempoolService`
- peer store: `PeerManager`
- wallet-local metadata: `WalletService`

The API must not write these stores directly. The GUI must not write them at all. The supervisor may coordinate process access but not mutate domain data.

Rebuildable versus canonical data

One of the most useful storage clarifications is to classify every dataset:

- canonical and irreducible
- canonical but replay-derivable
- rebuildable convenience index
- cache
- journal

Examples:

- canonical block and tip data: irreducible or at least primary
- UTXO state: canonical but replay-derivable
- private note state: canonical or replay-derivable depending on design, but treated as consensus-critical
- explorer indexes: rebuildable
- GUI query caches: rebuildable
- transaction lifecycle views: rebuildable from authoritative journals plus canonical events

This classification tells the team what may be optimized later, what must be preserved carefully, and what can be discarded or rebuilt after failure.

Revision markers

Repositories should expose monotonic revision markers for read models where feasible. At minimum:

- chain revision or tip revision
- mempool revision
- wallet-history revision
- peer-state revision

Why this matters:

- read consistency can be described and tested
- query caches can report freshness
- GUI state can reconcile events cleanly
- operators can diagnose stale or partial views

Revision markers are a cheap structural improvement with high value.

Caching strategy

Caches should become explicit rather than accidental.

Rules:

- every cache declares its source of truth
- every cache declares its invalidation triggers
- every cache declares whether stale reads are acceptable
- caches for consensus-critical domains are read-through views, not alternate mutation owners

Recommended cache classes:

- query-view caches
- API response caches
- wallet history composition caches
- peer summary caches

Avoid caches that:

- mutate canonical semantics
- outlive their source without revision checks
- are silently reused across network switches

Schema versioning and migrations

Deeper schema work belongs later, but the framework should be introduced earlier.

Needed components:

- per-store schema version tags
- network identity tags
- migration planner
- downgrade/rollback warning behavior
- backup-before-migrate hooks

Why this matters now:

- production upgrades eventually require it
- without schema metadata, restart and upgrade bugs become much harder to diagnose

However, the actual migration machinery should come after repository extraction, because otherwise the team will be versioning a structure that is still too diffuse.

Replay and recovery boundaries

Storage refactoring must respect replay semantics.

Recommended rule:

- replay/recovery logic reads and writes through the same repositories used by live acceptance wherever possible

That does not mean the code paths must be identical, but it does mean they must share the same invariants, write contracts, and validation assumptions.

Any repository API that cannot support both live and replay reasoning safely is not a mature repository boundary yet.

Avoiding direct LMDB access

The short-term goal is not to eradicate every direct LMDB call immediately. It is to stop new direct LMDB spread and progressively shrink the existing surface.

Safe first wraps:

- peer store access
- query-heavy reads
- wallet metadata reads
- mempool reads

Later wraps:

- chainstate reads used by API and explorer surfaces
- bond/stake queries
- private-state queries

Highest-risk last wraps:

- consensus-critical write paths

This preserves production safety while still moving toward cleaner storage architecture.

Query-read models as first-class storage consumers

`QueryService` should consume repository read interfaces rather than reaching through arbitrary business logic layers. That means the storage strategy is not just about writes. It is also about enabling consistent read views without letting query code mutate state.

This is one reason to add snapshot-capable or revision-labeled read methods early.

What must never be directly touched by GUI or API

- LMDB environments
- write transactions for consensus stores
- mempool write operations
- bond/stake mutations
- private-state mutations
- peer scoring writes
- recovery journals

The API should call services. The GUI should call the API. Neither should know how a store is laid out on disk.

Storage refactor conclusion

Storage refactoring should begin with repository boundaries, single-writer declarations, and revision-bearing read contracts. It should not begin with deep write-path redesign. That sequencing gives the team clearer interfaces and safer observability while protecting the most dangerous consensus-sensitive code from premature churn.

Repository method contract guidance

Repository methods should make semantics obvious.

Recommended categories:

- `get_*` for read-only retrieval
- `scan_*` for bounded iteration
- `put_*` only for canonical writes owned by the service
- `delete_*` only where the domain explicitly allows deletion
- `snapshot_*` for revision-bearing read views
- `inspect_*` for debug or invariant use

Every write-oriented method should declare:

- required ownership context
- expected preconditions
- revision behavior
- error codes on conflict or mismatch

This makes repositories safer to use and easier to mock in tests.

Backup and restore strategy

Storage cleanup should also define what a safe backup and restore story looks like.

At minimum, the system should distinguish:

- canonical chainstate and block data
- wallet secrets and wallet-local metadata
- rebuildable indexes and caches
- journals needed for restart diagnosis

Production guidance should eventually state:

- what must be backed up
- what can be rebuilt
- what happens if a user restores older wallet state over newer chainstate
- what happens if network identities differ

This is partly an operations topic, but it belongs in the storage strategy because repository and schema boundaries determine what restore safety even means.

Observability and Debuggability Refactor Plan

Why observability comes before deep refactoring

The previous audits repeatedly converged on the same conclusion: the system has real machinery, but too many dangerous transitions are still hard to prove from runtime evidence alone. That is why observability is not a support task. It is an enabling refactor. Without it, the team cannot confidently answer:

- which component actually accepted a transaction
- which revision the GUI was displaying
- whether a block was applied live or reconstructed after recovery
- whether a network switch fully completed before commands resumed
- whether a process restart created divergent state or merely reloaded the same state

Deep refactors performed without those answers are high-risk by definition.

Observability design goals

The observability program should make it possible to prove:

- lifecycle ordering
- ownership of mutations
- read freshness
- tx and block state transitions
- reorg effects across domains
- recovery equivalence
- source of stale or conflicting UI state
- cause of overload or degraded service

That means logs alone are not enough. The system needs structured events, correlation identifiers, revision markers, and selective trace surfaces.

Correlation IDs

Every meaningful action should have a stable correlation id.

At minimum:

- API requests

- lifecycle commands
- network switch operations
- wallet send intents
- built transactions
- mempool admission attempts
- block acceptance sequences
- reorg sequences
- replay/recovery runs

Where it should be added:

- API middleware
- supervisor command surfaces
- wallet service intent creation
- consensus block-accept entrypoints
- mempool admission path
- reorg/recovery entrypoints

Why it matters:

- allows cross-log reconstruction of a single operation
- makes ambiguous user bug reports actionable
- supports deterministic regression capture

Prerequisite status:

- required before medium-risk structural refactors land broadly

Structured logs

Plain string logs are not sufficient for this system.

Every critical log line should have structured fields such as:

- event type
- correlation id
- network
- role
- tip height
- state revision
- tx id or block hash where relevant
- lifecycle state
- error code
- latency

Where to add them first:

- supervisor lifecycle actions
- consensus block acceptance and reorgs
- wallet send flow
- API request/response summaries
- network switch flow

Why it matters:

- machine-queryable logs are essential once the system spans multiple processes and domains
- they make production diagnosis and trace stitching feasible

State transition logs

The most important logs are not generic debug statements. They are explicit stat

e-transition logs.

Required domains:

- process lifecycle transitions
- node readiness transitions
- transaction lifecycle transitions
- mempool admission and eviction transitions
- block acceptance phases
- reorg phases
- private-state update phases
- bond/stake update phases

What they should prove:

- the before state
- the requested transition
- the owner performing the transition
- the resulting state
- whether the change committed successfully

Without these logs, a later architectural cleanup can easily erase the team's ability to prove behavior that used to be visible only implicitly.

Startup dependency tracing

Startup tracing should answer:

- which services were built
- in what order
- with which network identity
- with which storage roots
- which dependencies were missing
- when readiness became true

This tracing is especially important because the system currently contains multiple entrypoints and overlapping runtime assembly paths. If the team wants to reduce that complexity safely, it first needs exact evidence about how the current runtime actually comes up.

Crash and recovery tracing

Recovery tracing should answer:

- what the system believed was last committed
- what journals or checkpoints were found
- whether replay ran
- what height or revision replay started from
- whether replay completed cleanly
- whether side-state rebuilds matched canonical expectations

Where to add it:

- `Src/Main/crash_recovery.py`
- consensus recovery entrypoints
- private-state rebuild paths
- bond/stake rebuild paths

This is launch-critical for any production system where restart safety matters.

Transaction lifecycle tracing

For every send intent, the system should log transitions such as:

- intent created
- preview generated
- tx built
- local validation attempted
- accepted or rejected by mempool
- relay attempted
- seen in block
- confirmed
- reorged out
- expired or failed

What it would prove:

- where a user-facing send actually failed
- whether a duplicate send is truly duplicate or merely ambiguous
- whether relay problems are being mistaken for wallet or mempool problems

This is one of the fastest ways to reduce support and debugging cost.

Reorg tracing

Reorg tracing is mandatory for production-grade consensus work.

Required trace points:

- competing chain summary
- fork point
- disconnected block list
- reconnected block list
- UTXO rollback status
- private-state rollback status
- bond/stake rollback status
- mempool repair summary
- final tip after reorg

What it would prove:

- whether all state domains participated in the reorg
- whether repair was partial or complete
- whether replay or recovery re-entered the process

This is essential before touching any reorg code.

Block acceptance tracing

Block acceptance should emit a stable phase model, for example:

1. received
2. header verified
3. txs verified
4. economic checks passed
5. chainstate commit started
6. chainstate commit complete
7. private-state hook complete
8. bond/stake hook complete
9. mempool cleanup complete
10. tip advanced

The exact phases may differ, but the principle matters. A future maintainer should not have to read several modules to discover where acceptance failed.

LMDB commit tracing

Storage commit tracing should record:

- repository or store name
- write domain
- correlation id
- revision before/after
- transaction duration
- commit success or failure

This should be selective and bounded so it does not create unacceptable overhead, but it is critical for diagnosing partial-failure and contention issues.

API request tracing

The API should log:

- request id
- endpoint
- auth scope
- latency
- backend service called
- success or failure code
- response revision metadata

This makes it possible to answer whether a stale GUI view came from stale data, a bad cache, or the GUI composing incompatible reads.

GUI event-consumption tracing

The GUI should trace, at least in debug mode:

- event received
- event revision
- store slices updated
- refreshes triggered
- stale event discarded

Why it matters:

- event-driven UIs often fail in subtle ways
- without client-side trace points, backend logs alone cannot explain stale or oscillating displays

Health-check traceability

Health and readiness decisions should themselves be observable.

The system should log:

- why a role is not ready
- which dependency failed readiness
- why a health check degraded
- when a role moved from ready to degraded

This is essential for supervisor correctness and operator trust.

Cache invalidation tracing

Cache bugs are often invisible because the system does not log invalidation reas

oning. Add trace points for:

- cache refresh
- cache invalidation cause
- cache reused with revision match
- cache bypass due to staleness

This should exist in:

- query service
- API response caches if any
- GUI state store

Metrics and dashboards

Logs and traces are necessary, but production hardening also requires metrics th
at can be watched continuously.

Recommended metric families:

- lifecycle metrics
  - process start latency
  - readiness latency
  - restart count
  - crash-loop count
- consensus-adjacent metrics
  - block acceptance latency
  - reorg count and depth
  - replay duration
  - mempool admission latency
- wallet and tx metrics
  - send intent creation count
  - send failure count by category
  - pending lifecycle age
- API metrics
  - request rate by endpoint family
  - error rate by code
  - rate-limit and overload responses
- GUI-client metrics in debug or optional telemetry mode
  - event reconnect count
  - stale-state reconciliations
  - failed command retries

Dashboards should group these around operational questions:

- is the backend healthy
- is the backend ready
- are sends failing locally or on relay
- are reorgs or replay events occurring more than expected
- is the GUI receiving coherent updates

Alerting thresholds

Before mainnet-grade operation, define at least soft alerts for:

- repeated crash restarts
- readiness never achieved within threshold
- repeated network-switch failures
- replay duration beyond expected range
- mempool admission failures spiking
- proof verification failures spiking
- reorg depth above expected norm

- API overload or event-stream saturation

Alerts do not need to page a full NOC from day one, but the thresholds and the instrumentation should exist.

Trace sampling and retention

Because full tracing of every path may become expensive, the observability plan should define:

- always-on low-cost structured logs for critical state transitions
- sampled deep traces for high-volume paths
- forced full traces for explicitly debug-enabled correlation ids
- retention rules that preserve critical failure traces longer than routine success traces

This avoids the common failure mode where observability is so verbose it becomes unusable, or so sparse it becomes irrelevant.

Observability rollout order

The recommended order is:

1. correlation ids
2. structured logging
3. lifecycle and tx/block state-transition logs
4. startup and recovery tracing
5. event-stream and GUI event tracing
6. selective LMDB and cache invalidation tracing

This order produces value quickly and supports the rest of the roadmap.

Instrumentation ownership

Observability work also needs explicit owners, otherwise it will become partial and inconsistent.

Recommended ownership:

- supervisor lifecycle traces: control-plane owner
- API request traces and error-code metrics: API owner
- wallet and tx lifecycle traces: wallet-service owner
- mempool admission and relay traces: mempool owner
- consensus, reorg, and replay traces: consensus owner
- repository commit and cache traces: storage owner
- GUI event and stale-state traces: GUI/store owner

This matters because observability is architecture. If every team adds traces in a different style and with different identifiers, the result will still be hard to use under production stress.

Observability conclusion

Observability is a prerequisite layer for the production refactor program. It reduces guesswork, lowers rollback difficulty, and turns future consensus-sensitive changes from speculative surgery into measurable engineering work.

Testing and Verification Refactor Plan

Testing philosophy

The system does not merely need more tests. It needs the right test layers, alig

ned with the actual architecture and failure modes. The current risk is not only unit-level logic bugs. It is multi-layer behavior bugs:

- startup ordering
- runtime authority overlap
- transaction lifecycle ambiguity
- reorg and replay drift
- private/public coexistence edge cases
- GUI/backend stale-state mismatches

The testing plan therefore needs to prove both correctness and architecture.

Tests to add before major refactoring

Before medium-risk refactors proceed broadly, add:

- API contract tests for all core node and wallet endpoints
- lifecycle command idempotency tests
- readiness versus liveness tests
- revision-consistency tests for key query endpoints
- GUI store tests for stale data and event reconciliation
- transaction-lifecycle tests for send, failure, and retry behavior

These tests stabilize boundaries so the team can change internals more safely.

Unit-test priorities

Add or strengthen unit tests for:

- error taxonomy mapping
- response envelope generation
- event payload schemas
- repository interfaces
- wallet intent state transitions
- GUI store reducers or mutation methods
- supervisor state transitions
- peer scoring decisions
- cache invalidation helpers

These are inexpensive compared with system tests and support the early-phase refactors directly.

Integration-test priorities

Integration tests should focus on domain boundaries:

- `WalletService` plus `MempoolService`
- `NodeService` plus `SupervisorService`
- `APIGateway` plus service container
- `QueryService` plus repositories
- `GUIStateStore` plus mock event stream

The key is to validate interfaces rather than only individual functions.

Multi-process tests

Because the system is runtime-topology-sensitive, multi-process tests are mandatory.

Required scenarios:

- start full node and API, verify readiness gating

- start miner alongside node, verify role status consistency
- stop and restart roles independently where allowed
- network switch under supervisor control
- verify writer-lock or single-writer discipline on shared storage roots

What these prove:

- process topology is real, not assumed
- lifecycle ownership is actually centralized
- runtime behavior matches the intended architecture

Lifecycle tests

The control plane requires dedicated lifecycle suites:

- cold start
- warm restart
- crash restart
- degraded startup
- partial dependency failure
- stop during busy state
- duplicate start and duplicate stop commands

Each should verify:

- command responses
- supervisor state transitions
- API readiness
- event emission
- final process state

Transaction lifecycle tests

For production stability, the send path needs its own explicit suite.

Required cases:

- public send success
- private send success
- duplicate submit with same idempotency key
- timeout then retry
- local validation reject
- mempool reject
- relay failure after local acceptance
- block inclusion
- confirmation
- reorged-out transaction
- restart during pending state

Each test should assert:

- backend lifecycle record
- API envelopes
- event sequence
- GUI store reaction in mock client tests

Block acceptance and reorg tests

Even if the roadmap avoids deep consensus rewrites at first, the test harness ne eds to improve early.

Required suites:

- valid block extend
- invalid header reject
- invalid tx reject
- reward overclaim reject
- mixed public/private block accept
- side-branch accumulation
- chainwork-favored reorg
- deep reorg with mempool repair
- reorg with bond/stake state rollback
- reorg with private-state rollback

Why these matter now:

- they provide a safety cage around adjacent refactors
- they reveal where current observability and authority models are too weak

Replay and recovery tests

These are especially important before touching storage or lifecycle code.

Required cases:

- clean restart without replay
- restart requiring replay
- restart after partial previous shutdown
- crash during block acceptance simulation
- crash during reorg simulation
- replay equivalence versus live-apply chain
- replay with private-state rebuild
- replay with bond/stake rebuild

Each should compare:

- tip height and hash
- UTXO summaries
- private-state revision or root
- bond/stake summaries
- state-hash where available

Private transaction tests

Given the prior audit findings, this area needs special emphasis.

Required suites:

- proof verifier unavailable must fail closed
- tx-body mutation invalidates proof
- malformed proof reject
- nullifier reuse reject
- note amount and hidden/public conservation checks
- block inclusion re-verifies proof
- reorg restores private-state correctness
- replay rebuilds private state exactly

These are mandatory before any private-pipeline refactor beyond wrappers and obs
ervability.

Bond/stake and miner-eligibility tests

Required suites:

- bond creation and activation
- eligibility at boundary heights
- cooldown or unlock if present
- slashing/penalty replay if present
- payout-set determinism
- reorg across activation windows
- replay across payout windows
- invalid eligibility block rejection

The purpose is to turn a complex, high-risk subsystem into a testable contract before it is reshaped.

API contract tests

Every core endpoint should have contract coverage for:

- response envelope fields
- error codes
- auth scope behavior
- revision metadata
- idempotency behavior where applicable
- deprecation compatibility for old routes

This is what allows the GUI and other clients to be simplified without fear of silent breakage.

Event ordering tests

If the event stream becomes important, ordering and reconciliation tests are mandatory.

Required cases:

- ordered tx lifecycle events
- out-of-order event arrival
- duplicate event arrival
- event gap requiring query refresh
- reconnect and resume
- stale event ignored due to lower revision

Without these tests, event-driven UI work is guesswork.

Storage isolation tests

Repository and control-plane changes should be backed by tests for:

- unauthorized writer rejection
- concurrent read with single writer
- network-root mismatch rejection
- migration version mismatch detection
- read cache invalidation after writes

These tests make the storage architecture enforceable rather than aspirational.

Test strategy by phase

Before refactoring:

- add contract and state-transition tests around current behavior

During refactoring:

- add interface-level tests for new services and repositories
- keep old and new paths running side by side where possible

Before high-risk changes:

- add equivalence and fault-injection tests
- require green replay/reorg suites

Continuous verification

The production-hardening branch should define a mandatory verification set including:

- core unit suites
- API contract suites
- lifecycle integration suites
- consensus regression suites
- private-pipeline suites
- replay/recovery suites

This should run on every major refactor touching shared boundaries.

Minimum production verification pack

Before the system is treated as production-ready after this refactor program, the team should be able to produce a verification pack containing:

- lifecycle command traces for start, stop, restart, and network switch
- normalized API contract results for node, wallet, query, and event endpoints
- send lifecycle traces for both public and private sends
- deep reorg and replay test results
- proof-path test results, including fail-closed cases
- bond/stake and miner-eligibility test results
- storage single-writer and migration-safety test results
- GUI stale-state and event-reconciliation test results

The pack should not be aspirational. It should be generated and archived for each major release candidate.

Release-blocking test categories

If any of the following categories is red, the associated phase should not ship:

- lifecycle authority tests
- response-envelope and auth-scope tests
- transaction lifecycle tests
- replay/recovery equivalence tests
- private proof and conservation tests
- bond/stake eligibility tests where affected
- GUI reconciliation tests where affected

This is the practical translation of “correctness over elegance.”

Test-data and fixture discipline

The test suite should also define stable fixtures for:

- short canonical chains
- reorg scenarios
- mixed public/private transaction sets
- bond activation and payout cases
- network-switch config sets

- restart and recovery cases

Good fixtures matter because they make differential testing and reproducibility much cheaper during refactor work.

Differential and shadow testing

For the most dangerous medium- and high-risk transitions, use differential strat egies instead of trusting single-path tests alone.

Recommended patterns:

- old and new query-service paths running side by side with result comparison
- old and new lifecycle status computation running side by side temporarily
- legacy and wrapped wallet send flows comparing lifecycle outputs
- replay-oracle comparisons for any consensus-adjacent internal cleanup

The point is not to keep duplicate systems forever. The point is to de-risk migr ation by proving equivalence before cutover.

Release-gate test matrix

Every refactor phase should define a required green set:

- Phase 0:
  - logging field tests
  - readiness status tests
  - response-envelope tests
- Phase 1:
  - GUI store tests
  - API contract tests
  - repository read tests
- Phase 2:
  - wallet/node service tests
  - tx lifecycle tests
  - peer and mempool boundary tests
- Phase 3:
  - multi-process lifecycle tests
  - network-switch tests
  - auth and idempotency tests
- Phase 4:
  - GUI event-ordering tests
  - stale-state reconciliation tests
  - send-flow UI integration tests
- Phase 5:
  - deep reorg suites
  - replay equivalence suites
  - economic and private-proof invariants
- Phase 6:
  - benchmark suites
  - overload tests
  - upgrade and restore tests

This matrix should become part of the branch policy, not an informal aspiration.

Evidence pack for risky merges

For any merge that changes:

- lifecycle ownership
- storage repositories used by canonical services
- transaction lifecycle behavior

- event ordering
- consensus-adjacent internals

require an evidence pack containing:

- test run summary
- trace samples
- rollback plan
- operator-visible behavior changes
- invariants affected

This forces architectural accountability and creates a durable review artifact for future debugging.

Testing conclusion

The testing plan is not subordinate to the refactor plan. It is part of the refactor plan. Every serious structural change in this roadmap either depends on a new test layer or exists to make a new test layer possible.

Optimization Strategy After Refactoring

Optimization sequencing principle

The system should distinguish sharply between:

- architecture work for correctness and maintainability
- performance work for speed and efficiency

In a system like this, premature optimization is dangerous for two reasons:

1. it tends to freeze the wrong boundaries
2. it often obscures the single-writer, lifecycle, and consistency issues that actually matter more for production safety

The rule should be simple: optimize only after the mutation and ownership model is clearer, unless there is a demonstrated operational emergency.

What performance work is safe now

Some performance work is low-risk and useful early:

- measure and profile startup time
- instrument endpoint latency
- instrument sync and query throughput
- optimize obviously wasteful GUI polling by replacing it with consolidated requests or events
- optimize read-only query composition if it reduces load without changing semantics

These changes improve operational clarity more than they change architecture.

What should wait

The following work should largely wait until boundaries are cleaner:

- mempool throughput optimizations
- aggressive cache layers over chainstate
- large sync pipeline parallelization
- miner template-selection optimization
- private proof generation pipeline changes
- deep LMDB layout tuning

- network serialization micro-optimizations

These optimizations are exactly the kind that can lock in problematic ownership patterns if done too early.

Mempool throughput

Optimize later, after:

- `MempoolService` owns admission cleanly
- tx lifecycle is explicit
- mempool policy versus consensus validity is better isolated

At that point the team can benchmark:

- admission latency
- conflict detection cost
- eviction cost
- relay batch behavior

without confusing throughput work with authority cleanup.

Sync speed

Optimize later, after:

- `PeerManager` and `SyncService` boundaries are clean
- replay/recovery behavior is instrumented
- query load is decoupled from hot write paths

Otherwise sync optimization can mask deeper control-plane or storage issues.

Database access patterns

Optimize later, after repository extraction gives the team a stable point to measure:

- repeated reads
- expensive scans
- cache miss patterns
- transaction contention

Repository boundaries make performance work safer because they create explicit choke points for profiling.

GUI responsiveness

Some improvements are safe now:

- state-store-based rerender control
- elimination of repeated widget-local requests
- event-driven refreshes

Heavier client-side caching or speculative UI logic should wait until backend contracts are stable.

Miner performance

Mining performance work should be separated into:

- PoW backend efficiency
- template-building efficiency

- candidate-selection efficiency

Only the first category is relatively safe early. The others depend on cleaner b
oundaries around miner service and consensus-sensitive template generation.

Private proof generation speed

This area should wait until:

- proof ownership is explicit
- verifier behavior is proven fail-closed
- statement construction is isolated

Optimizing a proof pipeline that still has correctness ambiguity is exactly back
wards.

Query indexing

Read-model and explorer indexing optimization becomes much safer after:

- `QueryService` exists
- repository read interfaces exist
- caches expose revision metadata

Then the team can add indexes and materialized views without risking write-path
confusion.

Resource contention optimization

Once runtime topology is cleaner, the team should measure and control:

- CPU contention between miner, node, API, and GUI
- RAM pressure from mempool and query caches
- file-handle pressure
- process restart storms under failure

This is one of the last optimization passes because it depends on the control pl
ane being trustworthy first.

Performance acceptance criteria by subsystem

Optimization work should not be considered successful just because a benchmark i
mproved. It should satisfy subsystem-specific acceptance criteria.

API/query acceptance criteria:

- latency improvements do not reduce revision fidelity
- overload protection remains correct
- command latency remains predictable under read load

GUI acceptance criteria:

- fewer requests and rerenders without added stale-state ambiguity
- event-driven updates do not create oscillation or duplicate-state bugs

Control-plane acceptance criteria:

- start and restart times improve without weakening readiness checks
- crash recovery remains explicit and observable

Storage acceptance criteria:

- improved throughput does not introduce hidden multi-writer behavior
- cache additions remain revision-aware

Consensus-adjacent acceptance criteria:

- any performance change touching tx admission, template generation, replay, or reorg paths must preserve all invariants and equivalence tests

These criteria keep the program from celebrating performance gains that actually erode trust or debuggability.

Optimization conclusion

Performance optimization is essential for production, but it is not phase one. The safest approach is to stabilize authority, interfaces, and observability first, then optimize through the new boundaries rather than around them.

Migration and Rollout Strategy

Migration philosophy

The migration strategy should minimize blast radius, preserve rollback options, and keep consensus risk low until the system has stronger guardrails. That means:

- additive changes before subtractive changes
- wrappers before rewrites
- side-by-side behavior comparison where possible
- explicit success criteria for each phase
- no deep consensus rewrite without tests and traces proving necessity

This is not a one-branch heroic refactor. It is a staged production program.

Phase 0: Observability and safety rails

Goals:

- make system behavior visible enough to support deeper work
- define ownership and invariants explicitly
- reduce debugging ambiguity immediately

Exact changes:

- add correlation ids to API, lifecycle, tx, block, and reorg paths
- add structured logs and normalized error taxonomy
- add readiness versus liveness distinction
- document current state ownership and single-writer assumptions
- add revision metadata to major query responses

Risks:

- log volume increase
- small changes to response shapes

Prerequisites:

- endpoint and lifecycle inventory
- agreement on error categories and event names

Test requirements:

- log field smoke tests

- API envelope tests
- readiness status tests

Rollback strategy:

- structured logs can coexist with old logs
- additive response fields can remain if rollback occurs

Success criteria:

- a single transaction, block acceptance, and restart can be traced end to end
- GUI can detect stale reads by revision
- supervisor or API can distinguish live from ready

Phase 1: Low-risk boundary cleanup

Goals:

- reduce authority ambiguity in product layers
- decompose giant files
- centralize read paths

Exact changes:

- split `Src/GUI/gui.py` into store, client, controller, and view modules
- split API routes from backend service calls
- add `QueryService` as a read-only façade
- wrap non-consensus LMDB access behind repositories
- add `SupervisorService` façade over current lifecycle logic

Risks:

- import and dependency regressions
- subtle GUI behavior changes

Prerequisites:

- phase 0 observability in place
- contract tests for endpoints and key GUI flows

Test requirements:

- GUI store tests
- API contract tests
- repository read tests
- lifecycle command smoke tests

Rollback strategy:

- keep old entrypoints delegating to new wrappers
- maintain compatibility shims for GUI and API until stable

Success criteria:

- GUI no longer performs direct orchestration
- API routes are transport-only or near-transport-only
- read-heavy queries come from one service surface

Phase 2: Service and interface extraction

Goals:

- make mutation ownership explicit
- isolate wallet, node, mempool, peer, and miner domains behind stable service surfaces

Exact changes:

- introduce `WalletService`, `NodeService`, `MempoolService`, `PeerManager`, `MinerService`
- move business logic out of API route handlers
- define repository interfaces for canonical stores
- normalize transaction-lifecycle records
- begin backend event bus implementation

Risks:

- service-surface churn
- duplicated logic during migration
- accidental boundary leaks if façades are incomplete

Prerequisites:

- phase 1 wrappers working
- clearer ownership docs

Test requirements:

- service interface tests
- tx lifecycle tests
- peer and mempool integration tests
- command idempotency tests

Rollback strategy:

- façades delegate to legacy code paths where necessary
- old endpoints remain available under compatibility mode

Success criteria:

- every major write domain has an identified owner
- GUI and API both depend on service surfaces rather than direct lower-level calls
- transaction lifecycle becomes backend-owned

Phase 3: Control-plane and API cleanup

Goals:

- centralize lifecycle authority
- remove API/bootstrap ambiguity
- harden readiness, restart, and network-switch behavior

Exact changes:

- make `SupervisorService` the only lifecycle authority
- add startup planner, readiness aggregator, and shutdown coordinator
- move network switching under supervisor ownership
- formalize API command/query/event families
- add auth scopes, rate limits, and idempotency keys

Risks:

- runtime topology regressions

- command timing changes
- operator workflow changes

Prerequisites:

- phase 2 services available
- observability of startup and restart sequences

Test requirements:

- multi-process startup tests
- restart and network-switch tests
- auth and rate-limit tests
- command retry tests

Rollback strategy:

- retain legacy entrypoints as supervisor delegates temporarily
- allow API compatibility mode for old clients while the new gateway stabilizes

Success criteria:

- GUI no longer guesses readiness
- lifecycle commands come through one authority
- mixed-network startup risk is materially reduced

Phase 4: GUI/state-model cleanup

Goals:

- make the frontend deterministic
- remove polling-driven guesswork where possible
- stabilize user-facing wallet and node flows

Exact changes:

- complete GUI state-store migration
- add backend event-stream consumption
- move send status, wallet status, and node status to revision-aware store slice
s
- isolate explorer bridge and web views
- remove remaining widget-level HTTP logic

Risks:

- UI regressions
- stale-event reconciliation bugs
- event-stream reconnect bugs

Prerequisites:

- phase 3 event and query contracts
- stable error taxonomy and revision markers

Test requirements:

- event ordering tests
- GUI store reconciliation tests
- send-lifecycle UI tests
- backend disconnect/reconnect tests

Rollback strategy:

- store can continue to support polling fallback until event path is proven
- old widget-local calls can remain behind feature flags temporarily

Success criteria:

- GUI renders backend truth rather than inferring it
- send flow and node-control flow are traceable and deterministic
- stale-state bugs become diagnosable rather than mysterious

Phase 5: Consensus-sensitive refactors only if evidence requires them

Goals:

- address any remaining consensus-core architecture risks that were proven by evidence, not by aesthetic preference

Exact changes:

- only the minimum necessary high-risk work, such as:
  - commit-order hardening
  - replay/reorg internal cleanup
  - private proof-path hardening
  - bond/stake replay or payout-path cleanup
- every change must be preceded by an explicit problem statement and equivalence harness

Risks:

- highest in roadmap
- direct consensus or recovery regression risk

Prerequisites:

- green replay, reorg, and equivalence suites
- structured consensus tracing
- clear evidence that the change is needed

Test requirements:

- fault injection
- differential acceptance tests
- deep reorg suites
- recovery equivalence tests
- economic invariant tests

Rollback strategy:

- feature-flag or branch-level isolation where possible
- maintain ability to revert to prior consensus path quickly before public rollout

Success criteria:

- proven reduction in ambiguity or correctness risk
- no loss of replay equivalence
- no new single-writer or recovery regressions

Phase 6: Optimization and production hardening

Goals:

- improve operational efficiency, packaging quality, monitoring, and deployment safety after boundaries are stable

Exact changes:

- query indexing and cache tuning
- sync and mempool performance improvements
- miner and proof-generation optimization where justified
- deployment packaging cleanup
- backup and restore tooling
- schema migration tooling
- monitoring dashboards and alert thresholds

Risks:

- performance work can still introduce subtle regressions
- packaging changes can affect rollout behavior

Prerequisites:

- stable service boundaries
- runtime topology clarity
- usable production telemetry

Test requirements:

- benchmark suites
- upgrade and migration tests
- backup/restore tests
- load and overload tests

Rollback strategy:

- keep performance features configurable
- deploy migration tooling conservatively

Success criteria:

- predictable operations under load
- repeatable upgrades
- measurable performance gains without ownership regressions

Environment rollout sequence

The phases above should also roll through environments deliberately:

1. local developer mode
2. deterministic CI environment
3. controlled integration environment with multi-process topology
4. staging or testnet environment
5. guarded production rollout

Why this matters:

- many failures in this roadmap will be topology- or timing-dependent
- they will not all appear in single-process local runs

Each environment should prove different things:

- local proves ergonomics and basic correctness
- CI proves repeatability
- integration proves runtime topology

- staging proves operational behavior over time
- guarded production proves real-world tolerance

Workstream coordination

This program should run as coordinated workstreams rather than as unrelated pull requests.

Recommended workstreams:

- observability and diagnostics
- API and contract cleanup
- GUI/store decomposition
- supervisor and runtime control
- repository extraction
- wallet/node service extraction
- consensus-proof and replay hardening

Each workstream should publish:

- active ownership assumptions
- changed interfaces
- blocked dependencies
- newly required tests

Without workstream coordination, two safe local refactors can easily create an unsafe combined topology.

Go/no-go criteria between phases

A phase should not advance if any of the following are true:

- ownership ambiguity increased instead of decreased
- rollback plan is unclear
- test coverage for the changed boundary is still missing
- observability for the changed boundary is weaker than before
- GUI or API must guess more, not less
- new multi-writer risk was introduced

This sounds obvious, but explicit gate rules prevent schedule pressure from pushing the project into unsafe sequencing.

Rollback doctrine

Every phase should define rollback in operational terms, not only git terms.

Examples:

- if the supervisor façade misbehaves, can operators still use legacy entrypoints safely
- if the GUI event path misbehaves, can the GUI fall back to polling without mutating behavior
- if a new query service returns inconsistent snapshots, can clients revert to older endpoints while preserving envelopes
- if a new service boundary causes send failures, can the backend delegate back to the legacy code path temporarily

Operational rollback is the real criterion here because a live codebase may have users or nodes relying on the changed behavior.

Change-window strategy

High-risk or high-blast-radius phases should land in smaller increments and calm er release windows.

Recommended rules:

- no large consensus-adjacent refactor bundled with GUI and API churn in the sam e release
- no network-switch behavior changes bundled with storage migration changes
- no tx lifecycle contract changes bundled with private proof pipeline fixes unl ess strictly necessary

Separating change windows reduces the number of interacting unknowns and makes b lame assignment tractable when something goes wrong.

Shadow-mode and canary strategy

For some medium-risk refactors, especially around reads, lifecycle status, and e vent models, a shadow-mode rollout is safer than immediate replacement.

Recommended uses:

- `QueryService` can run alongside existing query paths with result comparison
- new lifecycle-status computation can publish debug-only comparison output befo re becoming authoritative
- GUI event consumption can run alongside polling before polling is reduced
- repository wrappers can compare wrapped and legacy read results in test or deb ug builds

Canary rollout should be used when:

- a new service boundary changes behavior timing
- a new query model changes composition semantics
- the rollback path is available but expensive

This helps catch drift before the new path becomes the only path.

Suggested program timeline

If the team needs a practical sequencing heuristic, a reasonable program shape i s:

- weeks 1-3:
  - correlation ids
  - structured logs
  - revision metadata
  - ownership documentation
- weeks 3-6:
  - GUI file decomposition
  - API route extraction
  - response envelopes
  - error taxonomy
- weeks 6-10:
  - supervisor façade
  - readiness model
  - query service
  - repository read wrappers
- weeks 10-14:
  - wallet and node service façades
  - transaction lifecycle records
  - command/query endpoint families
- weeks 14-18:
  - event bus introduction

  - GUI store event reconciliation
  - lifecycle idempotency hardening
- weeks 18+:
  - only evidence-driven consensus-sensitive cleanup
  - performance and packaging work as boundaries stabilize

The exact dates will vary, but the dependency order should not.

Staffing and ownership model for the program

This roadmap is easier to execute if each stream has a clear technical owner:

- one owner for control plane and supervisor work
- one owner for API and service extraction
- one owner for GUI/store refactor
- one owner for repository and storage boundaries
- one owner for consensus tracing and replay verification

That does not mean siloed development. It means every stream has a person responsible for keeping the architectural contract coherent across many small PRs.

Required non-code deliverables per phase

Each phase should also produce a small but concrete documentation bundle:

- updated ownership map
- interface diff for changed services
- operational notes for new failure modes or rollback steps
- test additions and what they prove
- observability additions and how to read them

Without these deliverables, the project may merge a lot of technically good changes while still remaining difficult for future maintainers and operators to reason about.

Phase exit scorecards

Each phase should end with a short scorecard that answers:

- what ambiguity was removed
- what new owner became authoritative
- what behaviors are now observable that were not observable before
- what tests now exist that did not exist before
- what rollbacks were practiced or at least documented
- what still remains uncertain

Suggested scorecard examples:

Phase 0 scorecard:

- correlation ids present on lifecycle, tx, and API paths
- revision markers present on major query responses
- readiness status distinguishes live from ready

Phase 1 scorecard:

- GUI store exists and powers at least core screens
- API route handlers are thinner
- `QueryService` serves at least chain and wallet summary views

Phase 2 scorecard:

- wallet and node mutations route through service façades
- tx lifecycle record exists and is queryable
- repository seams exist for core read domains

Phase 3 scorecard:

- supervisor is sole lifecycle authority
- network switch routes through one orchestrator
- API command/query/event split is visible to clients

Phase 4 scorecard:

- GUI consumes events safely
- stale-state and reconciliation logic are test-covered
- node and send flows no longer rely on scattered polling guesses

Phase 5 scorecard:

- any consensus-sensitive changes are backed by equivalence proof
- replay and reorg evidence is stronger than before the change

Phase 6 scorecard:

- production monitoring, backup/restore, upgrade, and overload behavior are documented and test-backed

These scorecards give leadership and maintainers a way to measure actual architectural progress instead of just counting merged pull requests.

Rollout governance

Every phase should have:

- a defined owner
- a written success checklist
- a rollback checkpoint
- a verification run that includes both old and new concerns

No phase should be declared complete merely because the code merged. It should be complete when the intended architectural ambiguity actually decreased and the proof surface actually improved.

Migration conclusion

The safest route to production grade is phased, additive, and evidence-driven. The roadmap deliberately spends several phases reducing ambiguity outside consensus before it touches the deepest parts of consensus. That is the right tradeoff for a live blockchain codebase.

Refactoring Risk Register

Risk-register methodology

Each candidate refactor should be judged on:

- architectural value
- consensus sensitivity
- blast radius
- rollback difficulty
- required test burden
- operational impact if wrong

The key is not to avoid all risk. It is to choose risk in the right order.

Risk register table

Refactor	Area	Value	Risk	Blast radius	Rollback difficulty	Test burden	Production impact if wrong	Worth doing now
Structured logs and correlation ids	observability	very high	low	broad but additive	low	low	lower diagnosability only	yes
Response envelopes and error taxonomy	API	high	low	API and GUI	low	medium	client confusion	yes
GUI file decomposition	GUI	high	low	GUI only	low	medium	UI regression	yes
QueryService extraction	read model	high	low-medium	API, GUI, explorer	low	medium	stale or inconsistent reads	yes
Supervisor façade	control plane	very high	medium	node lifecycle, GUI, API	medium	medium	start/stop instability	yes
WalletService façade	wallet	high	medium	wallet, GUI, API	medium	medium	send regressions	yes
NodeService façade	runtime	high	medium	lifecycle and status	medium	medium	operational confusion	yes
Event bus introduction	state propagation	high	medium	backend and GUI	medium	high	duplicate or stale UI state	yes, after groundwork
Repository wrappers for canonical reads	storage	high	medium	API/query layers	medium	medium	stale data or read bugs	yes
MempoolService extraction	tx admission	medium-high	medium	tx pipeline and miner	medium	high	admission regressions	yes, after wallet/node services
PeerManager extraction	network	medium	medium	P2P and sync	medium	medium-high	peer instability	later
SyncService cleanup	network	medium	medium-high	sync, P2P, storage	medium-high	high	sync stalls or bad catch-up	later
MinerService extraction	mining	medium-high	medium-high	mining and tx selection	medium-high	high	template regressions	later
Consensus module decomposition	consensus	high	very high	whole node	high	very high	consensus split or replay divergence	not now
Reorg internals rewrite	consensus/recovery	high	very high	chainstate, private, bond/stake	very high	very high	catastrophic state divergence	not now
Private proof-path rewrite	private consensus	very high	very high	private tx and inflation safety	very high	very high	hidden inflation or tx invalidation bugs	only for proven issues
Bond/stake settlement rewrite	economics/eligibility	high	very high	mining, rewards, replay	very high	very high	reward or eligibility corruption	not now
Fork-choice rewrite	consensus	low	extreme	whole network	extreme	very high	consensus split	no unless bug proven
LMDB replacement	storage	low-medium	extreme	entire backend	extreme	extreme	data corruption or migration failure	no

Detailed risk commentary

Structured logging and tracing

Value:

- enormous leverage across every future change

Risk:

- low, mostly operational overhead and log-noise risk

Worth doing now:

- absolutely

GUI decomposition

Value:

- high maintainability gain
- fast reduction in bug density from clearer boundaries

Risk:

- mostly behavioral drift in UI timing and wiring

Worth doing now:

- yes, because the blast radius is contained and the structural payoff is immediate

Supervisor façade

Value:

- one of the highest in the roadmap because it addresses lifecycle authority directly

Risk:

- medium because lifecycle bugs can be very visible and disruptive

Worth doing now:

- yes, but only behind good startup and shutdown tracing

Wallet and node service extraction

Value:

- high because they eliminate scattered authority and support cleaner API and GUI work

Risk:

- medium because user-facing behavior flows through them

Worth doing now:

- yes, after low-risk wrapper work and contract tests

Event bus introduction

Value:

- high because it reduces polling and stale-state confusion

Risk:

- medium due to ordering and reconciliation hazards

Worth doing now:

- yes, but not as the first change; it depends on revision markers and GUI store cleanup

Repository wrappers

Value:

- high because they cap storage sprawl and enable testing

Risk:

- medium because storage abstractions can accidentally change behavior if they overreach

Worth doing now:

- yes, starting with read-heavy and non-consensus paths

Mempool extraction

Value:

- high enough to matter because admission ownership is central to send stability

Risk:

- medium because miner selection and relay depend on it

Worth doing now:

- yes, but only after transaction lifecycle and service boundaries are clearer

Consensus decomposition

Value:

- potentially high for long-term maintainability

Risk:

- too high for early work

Worth doing now:

- no

Reorg internals rewrite

Value:

- possibly high if real issues remain

Risk:

- among the highest possible in the codebase

Worth doing now:

- no

Private proof-path rewrite

Value:

- high if required to close correctness or security gaps

Risk:

- equally high

Worth doing now:

- only to fix clearly identified correctness failures, not for architectural tidiness

Risk-register conclusion

The register shows a consistent pattern: the highest-value early work is almost all boundary and observability work. The deepest consensus changes have high theoretical architectural value but poor near-term risk-adjusted value. That is exactly why they belong late or only under evidence-driven exception handling.

Risk decision rules

The risk register should drive actual decision-making. Recommended rules:

- do not approve a medium-risk refactor without named tests and rollback plan
- do not approve a high-risk refactor without explicit evidence that a lower-risk wrapper solution is insufficient
- do not approve any refactor that reduces observability on a boundary not yet stabilized
- do not batch multiple unrelated medium-risk refactors into one release unless they share the same verification surface

This keeps the roadmap disciplined when delivery pressure rises.

Stop signals for the program

The team should pause or slow the program if any of the following appear:

- replay or reorg equivalence starts flaking
- lifecycle authority becomes more ambiguous during a supposedly simplifying change
- a new service boundary increases the number of hidden cross-calls
- observability becomes noisier without becoming more useful
- GUI state bugs increase faster than backend ambiguity decreases
- rollback becomes harder while test evidence remains weak

These stop signals matter because refactor programs often fail not from one catastrophic mistake, but from continuing to push forward after the evidence quality starts degrading.

Risk ranking by architectural theme

Highest near-term risk if mishandled:

- lifecycle centralization without enough tracing
- send lifecycle normalization without idempotency and event discipline
- repository wrapping that accidentally changes canonical semantics

Highest absolute risk in the whole roadmap:

- reorg/replay internals
- private proof and hidden-value handling

- bond/stake settlement and payout logic
- commit-order changes

Highest risk of wasted effort:

- premature performance tuning
- broad miner redesign before service boundaries are stable
- broad sync redesign before control-plane cleanup

This ranking is useful because it separates “dangerous” from “expensive but low-value.”

What Should Not Be Refactored Yet

1. Fork choice

Do not refactor fork choice unless there is a proven correctness issue.

Reason:

- deterministic chainwork selection already exists
- the architectural payoff is low
- the consensus-split risk is extreme

The right action now is specification and tests, not redesign.

2. Monetary policy and emission logic

Do not “clean up” reward schedule, fee routing, burn logic, or payout calculations for style reasons.

Reason:

- these are consensus-critical economics
- the previous consensus review already treated them as production-sensitive
- any change belongs in the protocol domain, not a refactor bucket

If improvements are needed, they should be specification-hardening, tests, and observability first.

3. Reorg internals

Do not rewrite reorg application, rollback staging, or replay internals early.

Reason:

- blast radius is too high
- observability and equivalence coverage are not yet strong enough

Wrap and instrument first.

4. Private proof and hidden-value model

Do not perform broad architectural rewrites of the private pipeline until:

- proof ownership is pinned down
- existing failing proof-path tests are resolved
- replay and reorg behavior are better evidenced

Fix correctness bugs. Do not redesign for elegance yet.

5. Bond/stake settlement and payout generation

Do not heavily refactor the settlement path, activation path, or payout journal path until:

- the economic invariant suite is stronger
- replay equivalence is better proven

This area is too consensus- and economics-sensitive for early cleanup.

6. Canonical storage commit order

Do not reorder writes across chainstate, private state, and bond/stake state early.

Reason:

- even if the current order is imperfectly documented, it is a real production behavior
- reordering it can create crash and replay divergence

Add trace points and tests first.

7. Deep LMDB replacement or storage-engine swap

Do not consider replacing the storage engine until:

- repository boundaries exist
- actual storage limitations are measured
- migration and rollback tooling exist

This is a classic high-cost distraction if attempted too soon.

8. Large sync and P2P rewrites

Do not broadly redesign sync or networking while lifecycle and service ownership are still ambiguous.

Reason:

- runtime topology is the more immediate source of confusion
- P2P cleanup becomes much safer after control-plane centralization

9. Broad miner redesign

Do not rewrite miner candidate generation or miner/runtime topology early unless it is blocking production.

Reason:

- miner behavior touches mempool, payout logic, and consensus-sensitive template generation

The safer near-term work is to isolate `MinerService`, not to change core miner logic.

10. Major GUI redesigns unrelated to ownership

Do not spend time on sweeping visual or interaction redesign while the GUI still lacks a stable state-store boundary and normalized backend contracts.

Reason:

- it consumes effort without reducing the main structural risk

11. Clever optimization passes

Do not optimize hot paths just because they look expensive.

Reason:

- many hot paths are also ownership boundaries
- optimizing them before boundaries are clarified can preserve the wrong architecture and make later cleanup harder

What-not-to-touch conclusion

The theme is consistent: avoid deep changes in consensus-critical or topology-critical areas until wrappers, traces, and tests have reduced the uncertainty. The system does not need bravery first. It needs discipline first.

Final Production-Grade Refactoring Blueprint

Executive blueprint

The production-grade path for Atho is not a rewrite. It is a controlled narrowing of authority.

The system should move toward a shape where:

- consensus remains the protected center
- runtime lifecycle is owned by one supervisor
- wallet and node mutations are owned by real services
- API is a gateway, not a bootstrapper
- GUI is a client, not an orchestrator
- read models are separated from write models
- storage access is wrapped and classified
- observability and tests make the dangerous parts provable

That is the core strategy.

What to do first

Do first:

- add correlation ids, structured logs, and transition traces
- add revision markers to query responses
- define and document ownership boundaries in code
- split `gui.py` and centralize GUI state
- split API transport from backend logic
- create `QueryService`
- add `SupervisorService` façade
- normalize error taxonomy and response envelopes

Why:

- these changes reduce ambiguity immediately
- they are comparatively low risk
- they enable every more ambitious refactor later

What to do second

Do second:

- extract `WalletService`

- extract `NodeService`
- begin `MempoolService` boundary
- wrap repository access for read-heavy domains
- formalize command/query endpoint families
- add backend event bus and lifecycle records
- centralize lifecycle commands behind the supervisor

Why:

- these changes convert implicit coupling into explicit service boundaries
- they solve real operational and product bugs
- they are still safer than touching the consensus core directly

What to split

Split aggressively:

- GUI layers
- API routes versus services
- query/read models from write models
- lifecycle orchestration from domain mutation
- wallet UX logic from transaction validity logic
- peer management from consensus validation

Split conservatively:

- consensus internals
- reorg and replay internals
- private validation internals
- bond/stake replay and payout internals

The system needs uneven decomposition. Product layers need more splitting than c anonical-truth layers.

What to centralize

Centralize:

- lifecycle ownership in `SupervisorService`
- node-level commands in `NodeService`
- wallet mutation in `WalletService`
- mempool truth in `MempoolService`
- read composition in `QueryService`
- event propagation in `EventBus`
- GUI display state in `GUIStateStore`

Do not centralize into the API. Do not centralize into the GUI. Those are client and transport layers, not truth layers.

What to isolate

Isolate behind interfaces:

- storage repositories
- prover and verifier backends
- peer store and peer policy
- miner status and template-building surfaces
- private-state read models
- bond/stake read models

Isolation is most valuable when it prevents accidental cross-domain mutation or makes replay behavior easier to compare.

What to postpone

Postpone:

- consensus module rewrites
- commit-order changes
- reorg internals rewrites
- fork-choice changes
- large private-pipeline redesign
- large bond/stake settlement redesign
- deep storage-engine work
- aggressive performance tuning that depends on unclear boundaries

These are not forbidden forever. They are simply not phase-one work.

What to wrap before rewriting

Wrap before rewriting:

- current lifecycle entrypoints
- current API assembly paths
- current wallet send flows
- current repository access patterns
- current consensus acceptance outputs
- current recovery entrypoints

Wrapper-first refactoring preserves rollback options and creates stable seam lines for tests.

What parts should become smaller systems

These should become smaller, better-bounded systems:

- GUI
- API gateway
- wallet command flows
- node lifecycle control
- query and explorer surfaces
- peer and sync control

These should remain tightly controlled internal systems with carefully exposed seams:

- consensus acceptance
- reorg/replay machinery
- mixed public/private conservation
- bond/stake replay and payout logic

How to get to production grade safely

The safe route is:

1. make behavior visible
2. make ownership explicit
3. remove outer-layer authority overlap
4. isolate reads from writes
5. normalize lifecycle and transaction state
6. centralize control plane
7. harden service boundaries
8. only then touch consensus internals if the evidence still says they need to change

This order is not optional ceremony. It is what keeps a large refactor from becoming a consensus incident.

Final recommendation

If the team has limited bandwidth, the highest-risk-adjusted package to pursue immediately is:

- observability and traceability
- GUI decomposition and state-store extraction
- API gateway cleanup and response normalization
- supervisor ownership of lifecycle
- query-service extraction
- wallet and node service façades

That package will materially improve maintainability, debuggability, and production readiness without forcing a premature consensus rewrite.

Only after those land and the new tests and traces are green should the team revisit whether deeper consensus-sensitive refactors are actually required. It is entirely possible that once the surrounding architecture is cleaned up, the right answer will be to keep most of the current consensus core intact and simply harden its proof surface, tests, and recovery guarantees.

That is the production-grade blueprint: reduce ambiguity first, preserve canonical correctness, and spend invasive risk only where evidence proves it is necessary.

Production-readiness bar

The system should not be considered production-grade merely because it is cleaner. It should meet a real readiness bar:

- lifecycle ownership is centralized and observable
- API responses are normalized and revision-aware
- GUI is store-driven and event-aware, not orchestration-driven
- wallet send lifecycle is backend-owned
- repository boundaries exist for major stores
- single-writer assumptions are explicit and tested
- replay and reorg behavior have strong trace and test coverage
- private-proof failure modes are better pinned down than they are today
- rollback procedures exist for the major structural changes

That bar is demanding, but it matches the system’s risk profile.

What success looks like by domain

Success in this program is not only cleaner code. It is cleaner operation and clearer truth.

Consensus success looks like:

- fewer people needing to touch the core
- stronger replay and reorg evidence
- no accidental pressure to rewrite stable logic for outer-layer convenience

Wallet success looks like:

- one backend-owned send lifecycle
- fewer ambiguous failures
- no GUI-side status guessing

Control-plane success looks like:

- one lifecycle authority
- explicit readiness states
- predictable restart and network-switch behavior

API success looks like:

- boring, versioned, normalized contracts
- thinner routes
- clearer auth and overload behavior

GUI success looks like:

- one state store
- fewer ad hoc requests
- event-aware rendering grounded in backend revisions

Storage success looks like:

- repository boundaries
- clearer single-writer ownership
- better distinction between canonical data and rebuildable views

First concrete implementation tasks

If the team wants an immediate starting queue, these are the first concrete task s that align with the blueprint:

1. Add request ids and correlation ids to API middleware and lifecycle commands.
2. Define the standard API response envelope and implement it for status and wal let preview endpoints first.
3. Introduce a small `SupervisorService` façade that wraps current start/stop be havior without changing semantics.
4. Create `gui_store.py` and route one or two existing screens through it before splitting the rest of `gui.py`.
5. Create a `QueryService` for chain summary, wallet summary, and node status.
6. Add revision metadata to the new query responses.
7. Introduce typed result objects for lifecycle commands and send preview result s.
8. Add a `WalletService` façade that owns unlock and send-intent creation.
9. Add a basic event-stream endpoint carrying node status and tx lifecycle updat es.
10. Add integration tests for start/stop/restart and public send preview/submit.

Those ten tasks are intentionally chosen because they:

- reduce ambiguity immediately
- create reusable seams
- have manageable blast radius
- make later service extraction safer

What the team should be able to say after the first major milestone

After phases 0 and 1, the team should be able to say, truthfully:

- we know which service owns lifecycle and which service owns wallet mutation
- we can trace a send or restart from request to outcome
- the GUI is beginning to render revision-aware backend state instead of stitchi ng ad hoc reads
- API contracts are stabilizing instead of drifting

- storage reads are starting to funnel through explicit repository seams

If the team cannot say those things, it is too early to contemplate deeper consensus-sensitive refactors.

If those outcomes are visible, the refactor program is working even before the deepest consensus-sensitive work is reconsidered.

Recommended execution order in one view

```text
Phase 0  -> observability + ownership docs + revision markers
Phase 1  -> GUI/API decomposition + query service + supervisor façade
Phase 2  -> wallet/node/mempool/peer service façades + repositories
Phase 3  -> lifecycle centralization + API command/query/event cleanup
Phase 4  -> GUI event-driven state model + explorer isolation
Phase 5  -> only necessary consensus-sensitive hardening
Phase 6  -> performance, packaging, upgrades, backup/restore, monitoring
```

That ordering keeps the project aligned with the main principle of the earlier audits: the outer ambiguity is currently a bigger production risk than the mere existence of a large consensus core.

Final operating rule

For this codebase, every proposed refactor should be judged by one question:

Does this change make authority clearer without making canonical truth less provable?

If the answer is yes, it probably belongs in the roadmap. If the answer is no, or if the answer is uncertain, it probably needs more observability, more tests, or a safer wrapper-first approach before it should proceed.

Mainnet release gate after the refactor program

Before declaring the system production-ready after this program, the team should be able to answer yes to the following:

- Is there exactly one operational lifecycle authority?
- Is there exactly one backend owner for wallet send intent and unlock state?
- Is the GUI demonstrably a client over backend-owned truth rather than a fallback orchestrator?
- Do query endpoints consistently expose revision metadata and freshness semantics?
- Are major stores behind explicit repositories or service-owned access patterns?
- Can the team trace a send, a restart, and a reorg end to end by correlation id?
- Are replay and recovery equivalence suites green on the current release candidate?
- Are private-proof fail-closed cases green on the current release candidate?
- Are bond/stake and miner-eligibility regression suites green where touched?
- Does every medium- or high-risk refactor in the release have a rollback plan and evidence pack?

If several of those answers are still no, the system may be improved, but it is not yet at the production-grade bar defined by this document.

PR review checklist for this program

Every architectural PR in this program should answer, in its description:

- what boundary is being clarified
- what authority is being reduced or centralized
- what new owner is introduced or strengthened
- what tests prove the new behavior
- what observability proves the new behavior in runtime
- what rollback looks like
- whether any consensus-adjacent path is touched, directly or indirectly

This checklist is important because architecture quality often decays through many individually reasonable pull requests. The program needs a review discipline that keeps the big picture intact.

Six-month target architecture picture

If the roadmap is executed competently, the target picture six months out should look something like this:

```text
GUI
  -> GUIStateStore
    -> APIGateway
      -> WalletService
      -> NodeService
      -> QueryService
      -> EventBus transport

NodeService
  -> SupervisorService
    -> role runtimes

ConsensusEngine
  -> ChainstateManager
  -> PrivateStateManager
  -> BondStakeManager
  -> MempoolService

QueryService
  -> repository read interfaces

All canonical stores
  -> explicit repositories
```

In that picture:

- the GUI is smaller and more deterministic
- the API is thinner and more boring
- lifecycle is centralized
- read composition is explicit
- consensus remains authoritative but less entangled with product-layer churn

That is the architecture this roadmap is trying to reach without destabilizing the chain on the way there.

What should remain intentionally coarse-grained

A production-grade architecture is not the same thing as maximal decomposition. Some subsystems should remain intentionally coarse-grained because their correctness depends on tight coordination.

Those include:

- canonical block acceptance orchestration
- reorg and replay orchestration
- mixed public/private conservation enforcement
- bond/stake replay and payout application

The right refactor for these areas is usually:

- better seams
- better traces
- better invariants
- better tests

not uncontrolled fragmentation.

Final architectural doctrine

The doctrine emerging from the earlier audits and this blueprint is:

- canonical truth must stay tightly owned
- operational truth must become tightly owned
- read truth must become explicitly derived
- convenience logic must stop masquerading as authority
- performance work must follow boundary cleanup, not substitute for it

If the team holds to that doctrine, the codebase can become substantially more maintainable and production-ready without paying the unacceptable price of destabilizing consensus in the process.

What not to do under schedule pressure

The team should also be explicit about the shortcuts it must refuse even under deadlines:

- do not ship a lifecycle refactor without traceability because “it seems to work locally”
- do not merge a new service boundary that still exposes raw internals just to hit a milestone
- do not use the GUI to paper over missing backend lifecycle or transaction states
- do not hide failing proof or replay tests behind temporary skips without a bounded remediation plan
- do not batch multiple high-blast-radius changes to save release overhead

These rules matter because production incidents usually come from pressure meeting ambiguity.

Final scorecard for the whole program

At the end of the program, leadership should be able to grade the effort against five questions:

1. Is the system easier to reason about?
2. Is the system easier to debug in runtime?
3. Are the mutation owners and lifecycle owners now obvious?
4. Are the highest-risk consensus paths more provable than before?
5. Did the team achieve this without destabilizing canonical truth?

If the answer to those is yes, the program succeeded even if some ugly but stable consensus internals remain in place. In a blockchain system, that is not compromise. That is good engineering judgment.

Operational handoff checklist

Before the refactored architecture is treated as the new normal for operators and future maintainers, the team should complete an operational handoff checklist:

- supervisor runbooks updated for start, stop, restart, and network switch
- API contract reference updated for command, query, and event families
- GUI troubleshooting guide updated for stale-state and backend disconnect cases
- rollback playbooks updated for each medium- and high-risk architectural change
- backup and restore guidance updated to reflect repository and schema boundaries
- release engineering checklist updated for the new verification pack

This handoff matters because a production-grade architecture is only partly about code. It is also about whether future operators and contributors can use the system safely without rediscovering its rules from incident reports.

Steady-state target after the program

Once the roadmap is substantially complete, the steady-state engineering model should look like this:

- product-layer feature work usually touches GUI views, GUI store, API contracts, and query services
- operational changes usually touch supervisor, node service, and observability surfaces
- wallet behavior changes usually touch wallet service, transaction lifecycle, and query models
- consensus work is rarer, more isolated, and better protected by tests and traces
- storage work happens through repositories rather than through scattered direct store calls

That steady-state model is valuable because it improves team velocity. Engineers spend less time rediscovering hidden owners and more time making deliberate, reviewable changes.

Maintenance policy after the refactor program

A common failure mode after a successful architecture cleanup is gradual regression. To avoid that, the team should adopt a maintenance policy that preserves the gains:

- every new backend mutation feature must name its owner explicitly
- every new query surface must declare revision and freshness semantics
- every new long-running operation must expose lifecycle state or events
- every new storage access point must justify why it is not going through a repository or service interface
- every new GUI feature must go through the GUI state store rather than bypassing it with local transport logic

This policy turns the refactor program into a durable operating standard instead of a one-time clean sweep.

Code-review heuristics for the post-refactor architecture

Reviewers should ask a stable set of questions on every significant change:

- Did the change introduce a second owner for an existing mutation domain?
- Did it move any authority into the GUI or API that should remain in backend services?

- Did it make read consistency harder to describe?
- Did it increase or decrease rollback difficulty?
- Did it add observability proportional to the new complexity it introduced?
- Did it preserve the distinction between policy, convenience logic, and canonic
al truth?

These heuristics are especially important in a blockchain codebase because small
 ergonomic shortcuts can become long-lived architectural liabilities very quickl
y.

Twelve-month target behavior

Twelve months after the refactor program begins, the healthiest outcome is not t
hat the codebase has been endlessly restructured. The healthiest outcome is that
 the refactoring rate slows because the architecture is stable enough that routi
ne development no longer requires cross-cutting cleanup.

At that point, the team should ideally experience:

- fewer architectural regressions from routine feature work
- lower debugging time for wallet and lifecycle issues
- far less ambiguity during restart, upgrade, or network-switch investigations
- a smaller set of engineers needing to work inside the protected consensus core
- stronger confidence that consensus-sensitive work can be isolated, reviewed, a
nd tested deliberately

That is the strategic payoff this document is aiming for.

Governance for consensus-adjacent change after the refactor program

One of the biggest benefits of this roadmap should be that the team becomes more
 selective about when it touches the consensus core at all. After the program, c
onsensus-adjacent changes should face a higher bar than ordinary product work.

Recommended governance questions:

- Is the change fixing a proven correctness, safety, or operability issue?
- Can the issue be solved by a wrapper, interface cleanup, test addition, or tra
ce improvement instead of an internal rewrite?
- Has the current behavior been traced and documented well enough to define the
intended equivalence?
- Do replay, reorg, economic, and proof-related tests cover the affected path?
- Is the rollback story acceptable if the change behaves differently in producti
on?

If the answer to several of those is no, the change is probably too early regard
less of how desirable the resulting code shape appears.

Release-committee questions for major architectural changes

For any future release that includes medium- or high-risk architectural work, th
e final release review should ask:

- Which owners changed?
- Which boundaries became narrower and which became wider?
- Which previously ambiguous behaviors are now explicit?
- Which risks were accepted knowingly?
- Which tests and traces specifically justify that acceptance?
- Which old compatibility shims can now be removed, and which should remain long
er?

This review discipline is how the project stays deliberate after the initial har

dening effort is complete.

Long-term architecture review cadence

Once the refactor program has stabilized the codebase, the team should not return to ad hoc architecture review only when something breaks. A lightweight recurring review cadence is healthier.

Recommended cadence:

- monthly:
  - review new boundary violations or exceptions
  - review open compatibility shims
  - review observability gaps surfaced by incidents
- quarterly:
  - review service ownership map
  - review replay/reorg evidence quality
  - review whether any temporary wrappers should now be retired
- pre-release for major versions:
  - review lifecycle authority
  - review API contract drift
  - review storage schema and migration readiness
  - review consensus-adjacent changes against the governance questions above

The purpose is not ceremony. It is to prevent the architecture from slowly drifting back toward overlapping authority and implicit coupling.

Post-program health metrics

After the roadmap is substantially complete, the project should track a few architecture-health metrics, not only product metrics:

- percentage of API routes that are transport-only and delegate to services
- percentage of GUI state changes routed through `GUIStateStore`
- number of direct repository access points outside approved service layers
- number of lifecycle entrypoints outside `SupervisorService`
- number of compatibility shims still active
- number of known stale-state or ambiguous-send bugs reported per release

These metrics give the team a way to tell whether the architectural gains are holding over time.

Architecture-debt backlog discipline

Even after the main refactor phases, some known imperfections will remain. The team should manage them explicitly instead of letting them dissolve into general backlog noise.

Recommended backlog labels:

- `consensus-sensitive debt`
- `lifecycle authority debt`
- `read-consistency debt`
- `storage-boundary debt`
- `compatibility shim debt`
- `observability gap`

Every item in these categories should record:

- current owner
- why it was not solved earlier
- what evidence is missing

- whether it is blocked by tests, observability, or production timing

This keeps future cleanup work precise and prevents the project from forgetting why certain ugly but stable areas were intentionally left alone.

Operational success measures

In addition to architecture-health metrics, the team should watch whether day-to-day operations actually improve:

- fewer support incidents caused by ambiguous wallet state
- fewer “restart fixed it but we do not know why” cases
- fewer GUI/backend disagreement reports
- fewer manual workarounds for network switch or lifecycle operations
- faster mean time to diagnose send, sync, or restart issues

If those measures improve alongside cleaner code, the program is producing real production value rather than just cleaner repository structure.

Sunset criteria for compatibility shims

Compatibility shims are useful during migration, but they should not become permanent architecture.

Each shim should have explicit removal criteria such as:

- replacement path is fully covered by tests
- production traces show stable behavior for at least one release cycle
- operators no longer depend on the old path
- rollback no longer requires the shim

Keeping these criteria explicit prevents the codebase from carrying transitional complexity indefinitely.

Operator communication policy

Any release that changes lifecycle behavior, wallet flow, API contracts, or network-switch behavior should include operator-facing notes that state:

- what changed
- what stayed compatible
- what rollout precautions matter
- what rollback signs to watch for

This is part of production hardening because many operational incidents begin with silent behavioral drift rather than explicit breakage.

Documentation ownership

The refactored architecture will stay healthy longer if documentation has explicit owners too.

Recommended documentation owners:

- service ownership map: architecture owner
- API contract reference: API owner
- supervisor and lifecycle runbooks: control-plane owner
- wallet send and recovery behavior notes: wallet owner
- replay and consensus evidence notes: consensus owner

That avoids the common pattern where documentation decays because everyone assumes someone else is maintaining it.

Retrospective policy

After each major phase, run a short retrospective focused on:

- what architectural confusion was actually removed
- what new confusion accidentally appeared
- what tests or traces proved most valuable
- what should be done differently in the next phase

This keeps the program adaptive without letting it lose discipline.

Incident-review linkage

When production incidents happen after the refactor program, the team should connect them back to the architecture scorecards instead of treating them as isolated bugs. Each incident review should ask:

- did the issue cross an ownership boundary that should have been explicit
- did observability fail to prove the real sequence quickly enough
- did a compatibility shim, temporary wrapper, or old path still contribute to the failure

This ensures that the architectural improvements remain tied to real operational outcomes.

Stewardship note

The final responsibility of this program is stewardship. The team is not only cleaning code; it is deciding which parts of the system deserve stability, which parts deserve isolation, and which parts deserve stronger proof before anyone touches them again. In a blockchain system, that judgment is part of the architecture itself.

That is why the roadmap repeatedly chooses discipline over bravado: fewer hidden owners, fewer implied transitions, fewer unreviewable side effects, and fewer reasons for a routine product change to drift into protocol risk.

It also explains the central posture of the document: clean the outer layers first, make the runtime understandable, make the data paths explicit, and spend consensus risk only when the evidence says there is no cheaper safe path left.

If the team keeps that posture, future architecture decisions should become easier, not harder, because the system will offer clearer evidence about where truth lives, where complexity is justified, and where simplification is finally safe.

That, more than any diagram or naming scheme, is what separates a temporary cleanup from a durable production architecture that can absorb new features, new operators, and new stresses without losing its center.

Final close

The safest path to a production-grade Atho is not to make the system look perfectly modern on paper. It is to make the current system more legible, more deterministic, more observable, and more disciplined about ownership while protecting the protocol core from unnecessary turbulence. If the roadmap stays faithful to that principle, the codebase can become dramatically cleaner and easier to operate without paying for that improvement in consensus risk.